UNIVERSITE DE FIANARANTSOA

ECOLE DE MANAGEMENT ET D'INNOVATION TECHNOLOGIQUE

Mention : INFORMATIQUE

Parcours : Développement d'Application Intranet – Internet (D.A.2.I)

MEMOIRE DE FIN DE STAGE

Pour l'obtention du Diplôme de Licence en Informatique

Conception et réalisation d'une application de gestion des services dans une maison de production audio

Présenté par : RASOANIRINA Mbolatiana Marie Lucia

Sous l'encadrement : Monsieur RAZAFISAMBATRA Tommy Gabrielo

Année Universitaire : 2024-2025

CIRRUCULUM VITAE

PROFIL

Adresse

Lot 181-BII 3504-FR
Ankofafa Ambany

E-mail

luciarasoanirina8@gmail.com

Téléphone

0383970236

LANGUES

Malagasy : Langue maternelle Français : Niveau intermédiaire (B2) Anglais : Niveau intermédiaire (B1)

CONNAISSANCES

Python, HTML/CSS, Java, C#, PHP, SQLite, JavaScript

LOISIRS

Lecture : aventure, roman

Sports: Basket-ball, Volley-

RASOANIRINA MBOLATIANA MARIE LUCIA

Etudiante en Développement d'Application Intranet et Internet (DA2I) passionnée par le développement logiciel et les technologies innovantes.

EXPERIENCES EN 2023-2025

DEVELOPPEUSE EN JAVA (Projet étudiant) :

Développement d'une application bancaire en java swing.

DEVELOPPEUSE WEB (Projet étudiant) :

Développement d'une application gestion d'un garage, en front-end HTML/CSS/JavaScript et PHP en back-end, et utilisation de Xampp pour la partie base de données.

DEVELOPPEUSE EN C# (Projet étudiant) :

Développement d'une application de gestion d'un garage administratif.

DEVELOPPEUSE EN PYTHON (Projet étudiant) :

Développement d'une application sur la connaissance de la littérature.

FORMATION

Licence en Développement d'Application Intranet et Internet

-UNIVERSITE DE FIANARANTSOA

ECOLE DE MANAGEMENT ET D'INNOVATION TECHNLOGIQUE (EMIT) FIANARANTSOA

Niveau L1 : 2023-2024

Niveau L2 : 2024-2025

Baccalauréat scientifique, option C

-LYCCEE FO MASIN'I JESOA (FMJ) TALATAMATY FIANARANTSOA

AVANT-PROPOS

Ce mémoire de stage est rédigé dans le cadre de notre deuxième année de licence en informatique. Il retrace le déroulement et les résultats d'un stage effectué au sein de MERIX. Une structure spécialisée dans la création, la production, la diffusion et la gestion de contenus sonores. Ce stage a été une première immersion concrète dans le monde professionnel ; dont le thème est la conception et la réalisation d'une application de gestion des services dans une maison de production audio. Il s'est agi non seulement de mettre en œuvre les connaissances acquises au cours de notre formation, mais aussi de développer de nouvelles compétences techniques et méthodologiques en contexte réel.

En immergeant dans un environnement professionnel, nous avons eu une meilleure compréhension des enjeux liés à un projet informatique en contexte professionnel. Ce document présente l'ensemble des étapes suivies pour mener à bien ce projet, depuis la définition des objectifs jusqu'à la mise en œuvre de la solution.

REMERCIEMENTS

La réalisation de ce mémoire a été véritable parcours, enrichissant mais exigeant. Si ce travail a pu voir le jour, c'est grâce aux nombreux facteurs qui sont intervenus sur le plan moral, matériel et académique. Aussi, nous ne pouvons passer sous silence les soutiens et les collaborations précieuses qui nous ont été apportées tout au long de la réalisation de ce stage.

Ce chemin, nous ne l'avons pas parcouru seuls. A chaque étape, nous avons été guidés, soutenus, encouragés. C'est avec une profonde reconnaissance et une grande émotion que nous adressons nos remerciements à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce travail.

Nos pensées les plus sincères vont tout d'abord à :

- Professeur HAJALALAINA Aimé Richard, Président de l'Université de Fianarantsoa, Enseignant chercheur pour son appui constant au développement de l'enseignement supérieur et sa vision éclairée, qui inspire notre parcours ;
- Docteur HDR RAKOTONIRAINY Hasina Lalaina, directeur de l'Ecole de Management et d'Innovation Technologique (EMIT) Fianarantsoa, nous le remercions chaleureusement pour sa bienveillance, son ouverture et son confiance envers les étudiants ;
- Madame RABEZANAHARY Hobiniaina, Membre du Conseil d'Administration de l'Université de Fianarantsoa, nous tenons à vous adresser nos remerciements les plus respectueux, pour votre implication constante dans le bon fonctionnement de notre formation. Votre rigueur académique, votre bienveillance et votre disponibilité ont profondément marqué notre parcours. Merci pour l'attention que vous portez à chacun de vos étudiant et pour avoir facilité la concrétisation de ce mémoire ;
- Docteur TAREHY Brice Evrard, Responsable de Mention Informatique Niveau Licence, pour sa présence et sont accompagnement constant ;
- Monsieur RAZAFISAMBATRA Tommy Gabrielo, un Ingénieur en Informatique ; Responsable et Fondateurs de MERIX, nous exprimons notre reconnaissance pour son accueil chaleureux, sa collaboration et le cadre propice qu'il a mis à notre disposition qui ont rendu notre stage possible dans des conditions idéales ;

Nous souhaitons également exprimer notre reconnaissance à l'ensemble du personnel administratif, technique et enseignant de notre établissement ainsi que de l'entreprise

d’accueil. Merci pour votre disponibilité, votre gentillesse et votre implication discrète mais essentielle. Vous avez contribué à créer in environnement de travail motivant et respectueux, favorable à notre progression personnelle et professionnelle.

Enfin, comment ne pas rendre hommage à nos familles, nos piliers dans l’ombre. Vous avez été notre force dans les moments de faiblesse, notre courage dans les moments de doute, notre sourire dans les jours gris. Merci pour votre amour inconditionnel, votre soutien sans faille, et surtout, pour avoir cru en nous même lorsque nous doutions de nous-même.

Liste des cartes et figures

Figure 1.1 : Organigramme de l'EMIT 10
Figure 2.1 Logo de MERIX 11
Figure 6.1 : Modèle de Flux Conceptuel 34
Figure 6.2 : Modèle conceptuel de traitement 35
Figure 6.3 : Modèle conceptuel des traitement analytiques 36
Figure 6.4 : Modèle Organisationnel de Traitements 37
Figure 6.5 : Modèle organisationnel de traitement analytique 38
Figure 6.6 : Modèle conceptuel de données générale 40
Figure 6.7 : MCD côté sécurité 40
Figure : 6.8 Tables des données personnelles 41
Figure 8.1 Extrait de code de l'authentification côté frontend 64
Figure 8.2 Extrait du code de navigation 65
Figure 8.3 Extrait du code de l'accueil 65
Figure 8.4 Extrait de code représentant un CRUD 66
Figure 8.5 Extrait du code de l'exportation PDF 66
Figure 8.6 Extrait de code pour le thème 67
Figure 8.7 Extrait de code des fonctionnalités dans le paramètre 68
Figure 8.8 Extrait de code backend de la connexion 68
Figure 8.9 Extrait des CRUD côté backend 69
Figure 9.1 Interface authentification en mode clair 70
Figure 9.2 Interface de connexion en mode sombre 71
Figure 9.3 Recette annuelle et mensuelle 71
Figure 9.4 Interface représente le CRUD dans l'application 72
Figure 9.5 Extrait de la facture dans l'application 72
Figure 9.6 Extrait d'une facture en format PDF 73
Figure 9.7 Interface du paramètre 73
Figure : Image de Xampp 76

Liste des tableaux

Tableau 1.1 récapitulatif formation de l'EMIT 3
Tableau 6.1 Dictionnaire de donnée 28
Tableau 6.2 Dictionnaire de Traitement 31
Tableau 6.3 Modèle Organisationnel de Données 42
Tableau 7.1 Comparatif côté frontend 58
Tableau 7.2 Comparatif côté backend 60
Tableau 7.3 Comparatif des logiciels 62

Liste des abréviations

AES : Administration Economique et Sociale

API : Application Programming Interface

BD : Base de Données

CD : Déploiement Continu

CFC : Centre de Formation Continue

CI : Intégration Continue

CIGSI : Conception, Intégration et Gestion de Système d'information

CLI : Command Line Interface

CM : Communication Multimédia

CMW : Cultures et Métier du Web

CNMP : Communication Numérique et Management de Projet

CRA : Créer une Application React

CSS : Cascading Style Sheets

CUFP : Centre Universitaire de Formation Professionnalisant

DA2I : Développement d'Application Internet/Intranet

DOM : Document Object Model

E/A : Entité / Association

EMIT : Ecole de Management et d'Innovation Technologiques

ES : EcmaScript

G1 : Gestion - Secrétariat

G2 : Comptabilité Gestion

HMR : Hot Module Replacement

HTML : HyperText Markup Language

HTTP : HypertText Transfer Protocol

HTTPS : HypertText Transfer Protocol Secure

IA : Intelligence Artificielle

IGTI : Ingénierie Géo spatiale et Technologies de l'Information

JS : JavaScript

JSON : JavaScript Object Notation

JSX : JavaScript XML

L2 : Licence deuxième année

L3 : Licence troisième année

L.M.D : Licence, Master et Doctorat

M1 : Master première année

M2I : Modélisation et Ingénierie Informatique

MCD : Modèle Conceptuel de Donnée

MCF : Modèle Conceptuel de Flux

MCT : Modèle Conceptuel de Traitement

MCTA : Modèle Conceptuel de Traitement Analytique

MD : Management Décisionnel

MEDA : Management d'Entreprises et Développement des Affaires

MISS : Mathématiques et Informatiques pour les Sciences Sociales

MLD : Modèle Logique de Donnée

MLT : Modèle Logique de Traitement

MOD : Modèle Organisationnel de Donnée

MOT : Modèle Organisationnel de Traitement

MOTA : Modèle Organisationnel de Traitement Analytique

MPD : Modèle physique de Donnée

MVC : Model View Controller

NPM : Node Package Manager

PDF : Portable Document Format

PHP : HypertText Preprocessor

PM2 : Process Manager 2

REST : Representational State Transfer

RG : Règles de Gestion

RPCO : Relations Publiques et Communication Organisationnelle

S : Semestre

SDIA : Sciences des Données et Intelligence Artificielle

SGBD : Système de Gestion de Base de Donnée

SGBDR : Système de Gestion de Base de Donnée Relationnelle

SIGB : Système d'Information, Géomatique et Décision

TS : TypeScript

UF : Université de Fianarantsoa

UI : Interface Utilisateur

UML : Unified Modeling Language

URL : Uniform Resource Locator

UX : Expérience Utilisateur

VSC : Visual Studio Code

Glossaire

Maison de production audio : entreprise spécialisée dans la création, l'enregistrement, le mixage, la distribution et la gestion de contenus audio.

Application web : logiciel accessible via un navigateur web, développé pour être utilisé sur internet ou un intranet, sans nécessité d'installation locale.

Frontend : partie visible de l'application par l'utilisateur final (interface utilisateur ou côté client), développée avec des technologies comme HTML, CSS, REACT, JavaScript, etc.

Backend : partie invisible de l'application qui traite la logique métier, les requêtes, les connexions à la base de données, etc.

Base de données relationnelles : système de gestion de données organisé en tables liées entre elles par des relations (exemple : MySQL, PostgreSQL, Maria DB).

SGBDR (Système de Gestion de Base de Donnée Relationnelle) : logiciel permettant de gérer les bases de données relationnelles. Il offre des fonctionnalités comme l'ajout, la modification, la suppression et l'interrogation des données.

CRUD : acronyme de CREATE, READ, UPDATE, DELETE qui sont des opérations de base permettant de gérer les données ans une base de données.

API (Application Programming Interface) : interface permettant à des applications de communiquer entre elles, souvent via des requêtes HTTP pour une application web.

Session : mécanisme permettant de mémoriser des informations sur l'utilisateur entre plusieurs pages d'un site.

Authentification : processus de vérification de l'identité d'un utilisateur.

Formulaire : interface web permettant à l'utilisateur de saisir, de modifier des données dans une application.

Responsivité : technique de développement d'interface web qui s'adaptent à toutes les tailles d'écran.

UI (Interface Utilisateur) : ensemble des éléments graphiques aves lesquels l'utilisateur interagit.

UX (Expérience Utilisateur) : ressenti global de l'utilisateur lorsqu'il interagit avec l'application (fluidité, simplicité, rapidité).

Sécurité web : ensemble des techniques utilisées pour protéger les données de l'application.

Technologies web : ensemble des langages et des outils utilisés dans le développement.

Framework : outil ou environnement facilitant le développement en imposant une structure.

Merise II : méthodologie d'analyse et de conception des systèmes d'information, utilisée pour modéliser les données, les traitements et les flux d'information. Version améliorée de Merise avec une meilleure gestion du temps et des processus.

Table de matières

CIRRUCULUM VITAE i
AVANT-PROPOS ii
REMERCIEMENTS iii
Liste des cartes et figures v
Liste des tableaux vi
Liste des abréviations vii
Glossaire x
Table de matières xii
INTRODUCTION 1
Partie I : PRESENTATION GENERALE 2
Chapitre 1 : PRESENTATION DE L'EMIT 3
1.1 Historiques 3
1.2 Missions 4
1.3 Formations existantes 4
1.3.1 Cycle Licence 5
1.3.2 Cycle Master 6
1.4 Organigramme de l'EMIT 10
Chapitre 2 : PRESENTATION DE L'ENTREPRISE D'ACCUEIL 11
2.1 À propos de Merix 11
2.2 Nos Services 12
2.2.1 Production musicale 12
2.2.2 Publicité sonore et identité audio 12
2.2.3 Podcasts et contenus narratifs 12
2.3 Notre expérience et Notre Vision 12
2.4 Contact 13
Chapitre 3 : PRESENTATION DU PROJET 14
3.1 Origine du projet 14
3.2 Contexte du projet 14
3.3 Problématiques 15
3.4 Solution proposée 15
3.5 Objectif du projet 16
3.6 Conduite du projet 16

Partie II : Analyse et Conception du projet 17
Chapitre 4 : METHODES, NOTATIONS UTILISEES 18
4.1 Méthodes 18
4.1.1 Comparaison avec UML 18
4.1.2 Avantages spécifiques de Merise II dans ce projet 18
4.2 Notations utilisées 19
4.2.1 La modélisation conceptuelle 19
4.2.2 La modélisation logique 21
4.2.3 Modélisation organisationnelle 22
4.2.4 Modélisation physique 23
Chapitre 5 : ANALYSE DU PROJET ET ETUDE DE L'EXISTANT 25
5.1 Présentation de la structure 25
5.2 Description du système actuel 25
5.3 Analyse des besoins fonctionnels 26
5.3.1 Besoins principaux 26
5.3.2 Besoins non fonctionnels 26
5.4 Analyse des outils concurrents 26
Chapitre 6 : CONCEPTION DU PROJET 28
6.1 Dictionnaire des données 28
6.2 Dictionnaire de traitement 31
6.3 Modèle de flux conceptuel (MFC) 34
6.4 Modèle conceptuel de traitement (MCT) 35
6.5 Modèle conceptuel des traitements analytiques (MCTA) 36
6.6 Modèle organisationnel de traitements 36
6.7 Modèle organisationnel de traitements analytiques 37
6.8 Règle de gestion 39
6.9 Modèle conceptuel de donnée (MCD) 39
6.9.1 Gestion générale 40
6.9.2 Gestion sécurité 40
6.9.3 Les données personnelles 41
6.10 Modèle logique des données (MLD) 41
6.10.1 Modèle logique de données pour la gestion globale 41
6.10.2 Modèle logique de données sur la partie sécurité 42
6.10.3 Modèle logique de données sur les informations du studio 42
6.11 Modèle organisationnel de données 42
6.12 Modèle physique de données 43

6.12.1 Modèle physique de données clients 43
6.12.2 Modèle physique de données services 44
6.12.3 Modèle physique de données commandes 44
6.12.4 Modèle physique de données ligne de commandes 44
6.12.5 Modèle physique de données utilisateur 45
6.12.6 Modèle physique de données connexions 45
6.12.7 Modèle physique de données studio 46
Partie III : Réalisation du projet 47
Chapitre 7 : Spécification des outils de réalisation 48
7.1 Présentation des outils 48
7.1.1 Interface utilisateur 48
7.1.2 Backend 52
7.1.3 Autre outil 54
7.1.4 Base de données 56
7.1.5 Outils logiciels 57
7.2 Justification des choix 58
7.2.1 Frontend : TypeScript, React, Tailwind CSS, Vite 58
7.2.2 Backend: Node.js, Express.js, MariaDB 60
7.2.3 Logiciels : Visual Studio Code, Xampp, Google Chrome 62
Chapitre 8 : Mise en œuvre et implémentation 64
8.1 L'implémentation côté frontend 64
8.2 L'implémentation côté backend 68
Chapitre 9 : Présentation de l'application développée 70
9.1 Présentation de la page d'authentification 70
9.2 Présentation du page d'accueil 71
9.3 Présentation du CRUD 71
9.4 Présentation de facture 72
9.5 Présentation des fonctionnalités supplémentaires 73
CONCLUSION 74
BIBLIOGRAPHIE ET WEBOGRAPHIE 75
ANNEXES 76

INTRODUCTION

La transformation numérique désigne l'intégration stratégique des technologies digitales dans les processus organisationnels, dans le but d'optimiser les performances, d'améliorer et de favoriser l'agilité opérationnelle.

Dans un contexte où le numérique redéfinit les modes de fonctionnement des organisations, la mise en place de solutions logicielles sur les problèmes devient un levier essentiel de performance. Les entreprises, quel que soit leur domaine d'activité, sont de plus en plus amenées à repenser leurs processus internes pour gagner en efficacité, en traçabilité et en qualité de service. Le secteur de la production audio, souvent centré sur la créativité et la technique, n'échappe pas à ce besoin d'optimisation.

C'est dans ce cadre que s'inscrit le projet objet de ce mémoire, réalisé lors d'un stage de fin de deuxième année de licence en développement d'application intranet-internet cher MERIX. Il a consisté à concevoir et développer une application de gestion de services pour la maison de production audio, afin de faciliter la planification des prestations, la gestion des clients et le suivi des projets.

MERIX est une maison de production audio dynamique qui s'inscrit dans une démarche artistique et technique plaçant la qualité sonore au cœur de son identité. Dans le domaine de la production audio, où plusieurs prestations sont menées en parallèle, l'organisation et le suivi des tâches sont essentiels pour garantir la qualité et la fluidité du travail.

L'objectif principal était de développer un outil permettant à l'entreprise de centraliser ses informations, de suivre les prestations en cours, de gérer les clients et d'optimiser la planification de ses ressources. C'est dans ce concordance que s'intitule le projet présenté dans ce mini mémoire ayant pour thème : « Conception et réalisation d'une application de gestion des services dans une maison de production audio ».

L'architecture de ce mémoire se distingue en trois grandes parties : en premier temps la présentation générale, puis dans la deuxième partie analyse et conception du projet et enfin la troisième partie qui est consacrée à la phase de réalisation concrète du projet.

Partie I : PRESENTATION GENERALE

Chapitre 1 : PRESENTATION DE L'EMIT

L'Ecole de Management et d'Innovation Technologiques connue sur le nom de EMIT est une Université publique rattachée à l'Université de Fianarantsoa. Elle se distingue par une approche orientée vers la pratique qui a basculé totalement vers le système L.M.D. L'école forme des professionnels capables d'innover et de gérer des projets dans un environnement technologique en constante évolution, avec un fort ancrage local et une ouverture sur les défis globaux. L'EMIT nous a reconnu comme étudiants.

1.1 Historiques

L'EMIT est une école universitaire publique pluridisciplinaire, rattachée à l'Université de Fianarantsoa. La grande maturité au niveau de l'enseignement et la compétence des étudiants sortants de l'établissement ont permis aux dirigeants sous l'approbation du Ministère de mettre en place la conversion du Centre en Ecole au sein de l'Université de Fianarantsoa par le Décret N°2016-1394 du 15 Novembre 2016. L'EMIT prépare d'une part le diplôme de Master en deux mentions selon trois parcours et d'autre part le diplôme de Licence en trois mentions selon cinq parcours.

Auparavant, elle était connue sous le nom du Centre Universitaire de Formation Professionnalisant (CUFP), créée par le Décret N°2005-205 du 26 Avril 2005 et dispensait le diplôme de Licence professionnelle en Administration ainsi qu'en Informatique. Mais avant cela, elle était connue également sous le nom du Centre de Formation Continue (CFC), créé par l'Arrêté Rectoral N°99-23/UF/R du 10 Mars 1999 qui formait de diplôme de Technicien Supérieur.

L'EMIT a été sélectionnée « Meilleur Etablissement » pendant le Salon de la Recherche organisé par l'Organisation Internationale du Travail les 5 et 6 Juillet 2017. Depuis l'année universitaire 2013-2014, l'école est basculée totalement vers le système Licence, Master et Doctorat (LMD). Toutes les offres de formation dispensée à l'EMIT sont habilitées par le Ministère de l'Enseignement Supérieur et de la Recherche Scientifique. L'école propose neuf parcours répartis en trois mentions, Présentes ci-dessous :

Tableau 1.1 récapitulatif formation de l'EMIT

<table>
<tr><td>Cycle</td><td>Mention Management</td><td>Mention Informatique</td><td>Mention Relations Publiques et Multimédia</td></tr>
<tr><td rowspan="2">Licence</td><td rowspan="2">Administration Economique et Sociale</td><td>Développement d’Application Internet/Intranet</td><td>Communication Multimédia</td></tr>
<tr><td>Conception, Intégration et Gestion des Systèmes d’Information</td><td>Relations Publiques et Communication Organisationnelle</td></tr>
<tr><td rowspan="4">Master</td><td>Management Décisionnel (MD)</td><td>Ingénierie Géo spatiale et Technologies de l’Information (IGTI)</td><td>Communication Multimédia (CM)</td></tr>
<tr><td rowspan="3">Management d’Entreprises et Développement des Affaires (MEDA)</td><td>Modélisation et Ingénierie Informatique (M2I)</td><td>Relations Publiques et Communication Organisationnelle (RPCO)</td></tr>
<tr><td>Système d’Information, Géomatique et Décision (SIGD)</td><td rowspan="2">Communication Numérique et Management de Projet (CNMP)</td></tr>
<tr><td>Sciences des Données et Intelligence Artificielle (SDIA)</td></tr>
</table>

Source : Ecole de Management et d’Innovation Technologique

1.2 Missions

L’école a pour mission, d’abord de dispenser des formations initiales et continues en informatique, en administration et en relations publiques et multimédia. Ensuite, elle offre des services connexes à l’informatique. Puis, elle forme des techniciens opérationnels immédiatement au sein des entreprises. Enfin, elle assure le perfectionnement professionnel des étudiants, des demandeurs d’emploi, des employés et des cadres d’entreprises.

1.3 Formations existantes

L’école présente actuellement deux cycles : Licence et Master. Chaque cycle possède plusieurs parcours assurés par un chef de mention.

1.3.1 Cycle Licence

L'école possède trois (03) mentions pour cinq (05) parcours en cycle Licence. Les étudiants toutes mentions confondues doivent en faire, un stage de réalisation en entreprise avec un rapport de stage soutenu pour les étudiants en deuxième année de Licence (L2) et un stage de fin d'études suivi de la soutenance d'un mémoire pour les étudiants en troisième année de Licence (L3). La formation dure trois années universitaires et à la fin de la formation, les étudiants obtiennent des diplômes de Licence.

1.3.1 -a Mention Management

Parcours Administration Economique et Sociale (AES)

La condition d'accès en première année de Licence se fait par voie de concours pour les titulaires d'un Baccalauréat général de toutes séries (A, C, D, S, L, OSE) ou d'un Baccalauréat technique G1 et G2. A l'issue de la formation, les étudiants ont les compétences de :

- Assister le Directeur Général, le Directeur des Ressources Humaines et le Directeur
- Administratif et Financier ;
- Gérer les Ressources Humaines ;
- Gérer une entreprise ou un projet.

1.3.1 -b Mention Informatique

Parcours Développement d'Application Internet Intranet (DA2I)

Parcours Conception, Intégrations et Gestion des Systèmes Information (CIGSI)

La condition d'accès en première année de Licence se fait par voie de concours pour les élèves titulaires du diplôme de Baccalauréat Général Scientifique (Série C, série D et série S), Baccalauréat Technique Professionnelle et Baccalauréat Technique Technologique (Filière Industrielle, Maintenance Automobile, Ouvrage Bois, Ouvrage Métallique, Génie Civile). A l'issue de la formation, les étudiants sont compétents en :

- Administration des bases de données ;
- Administration des réseaux et systèmes informatiques ;
- Développement d'application client/serveur

1.3.1 -c Mention Relations Publiques et Multimédia

Parcours Communication Multimédia (CM)

Parcours Relations Publiques et Communication Organisationnelle (RPCO)

La condition d'accès en première année de Licence se fait par voie de concours pour les titulaires d'un Baccalauréat général de toutes séries (A, C, D, S, L, OSE) ou d'un Baccalauréat technique G1 et G2. A l'issue de la formation, les étudiants ont les compétences de :

- Rédiger un article dans un journal ;
- Occuper un poste d'un technicien de presse ;
- Travailler dans la revue de presse.

1.3.2 Cycle Master

L'Ecole de Management et d'Innovation Technologique (EMIT) possède deux (03) mentions pour trois (09) parcours en Master Professionnel ainsi qu'en Master Recherche. La durée de la formation est de quatre semestres c'est-à-dire deux années universitaires.

1.3.2 -a Mention Management

Parcours Management Décisionnel (MD)

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Administration Economique et Sociale, en Gestion ou en Economie.

Le master en management décisionnel propose une formation approfondie et pluridisciplinaire, façonnant les étudiants dans les sphères de l'administration privée et publique. En mettant l'accent sur la stratégie décisionnelle, sur l'analyse de l'environnement d'intervention, la gestion des risques, les diplômés sont armés pour aborder des défis complexes. L'expertise des étudiants est fortement sollicitée dans divers secteurs, allant de la gestion d'entreprise à l'administration publique, où ils jouent un rôle crucial dans la prise de décision. En combinant une compréhension approfondie des disciplines et des compétences aiguisées en matière de décision, ce master offre une contribution significative à la réussite des organisations dans un environnement dynamique.

1.3.2 -b Management d'Entreprises et Développement des Affaires (MEDA)

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Administration Economique et Sociale, en Gestion ou en Economie.

Le master en management d'entreprises et développement des affaires prépare les étudiants à exceller dans la création et la constitution d'entreprises. Les diplômés acquièrent des compétences clés pour élaborer des plans d'affaires solides, identifier les opportunités de marché et gérer efficacement les ressources. Les étudiants sont bien positionnés pour fonder leur propre entreprise en tant qu'entrepreneurs prospères ou occuper des rôles clés dans des start-ups. Les débouchés incluent la création d'entreprises innovantes, la gestion de projets entrepreneuriaux et la consultation stratégique pour les nouvelles initiatives commerciales. Ce master offre ainsi une voie dynamique vers l'entreprenariat et le succès dans le domaine des affaires.

1.3.2 -c Mention Informatique

Parcours Système d'Information, Géomatique et Décision (SIGD)
Parcours Modélisation et Ingénierie Informatique (M2I)

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Informatique et en Mathématique et Informatique pour les Sciences Sociales (MISS). Le Recrutement en S9 se fait par validation des crédits acquis.

Le parcours « Système d'Information, Géomatique et Décision » a pour objectif de donner un panorama des recherches actuelles et émergeantes en termes de système d'aide à la décision. En effet, les systèmes informatiques et la géomatique sont en plein essor, par les grilles de calcul et les multiples appareils mobiles intégrant des systèmes informatiques de plus en plus performants et complexes. Ces systèmes informatiques intégrant un parallélisme massif ou /et une mobilité des composants représente un défi pour le génie logiciel qui doit fournir de nouvelles méthodes et des outils de production de logiciel pour la description de l'architecture de ces systèmes complexes et pour leur validation et/ou certification. De plus, l'effervescence des techniques en géomatique qui fournissent des données spatiales et

temporelles dans différents domaines représente des moyens efficaces pour prendre les bonnes décisions.

1.3.2 -d Mention Informatique IGTI /SDIA

Parcours Ingénierie Géo spatiale et Technologies de l'Information (IGTI)

Parcours Sciences des Données et Intelligence Artificielle (SDIA)

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Informatique et en Mathématique et Informatique pour les Sciences Sociales (MISS). Le Recrutement en S9 se fait par validation des crédits acquis.

Ce parcours vise à former des experts en informatique de haut niveau, spécialisés en intelligence artificielle et science des données. Les étudiants développeront des compétences clés telles que la conception et le développement de logiciels pour l'analyse automatique de grandes masses de données, la programmation du Big Data et de l'IA, la modélisation et l'automatisation du raisonnement sur les données, la conception d'architectures d'entreprise, l'application des méthodes statistiques, la maîtrise des techniques de gestion d'entreprise, l'utilisation de technologies pour l'exploitation de données réelles.

Ce parcours met l'accent sur le développement de solutions en géomatique en utilisant une expertise avancée dans la conception de systèmes d'information géographique, la télédétection et l'analyse spatiale. Les étudiants qui le suivent sont aptes à proposer des solutions informatiques adaptées aux besoins des utilisateurs en géomatique.

1.3.2 -e Mention Relations Publiques et Multimédia CM

Parcours Communication Multimédia (CM)

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Communication Multimédia (CM)**.** Le Recrutement en S9 se fait par validation des crédits acquis.

Le secteur de la communication multimédia est un secteur extrêmement varié et concurrentiel. Cette formation s'appuie par exemple, sur la recherche et traitement d'information avant la mise en publication (dans un site web, radio, télévision).

Le double diplôme entre le parcours Cultures et Métiers du Web (CMW) de l'Université Gustave Eiffel et le parcours Communication Multimédias (CM) de l'Université de Fianarantsoa propose l'obtention simultanée d'un diplôme de master en CMW et d'un diplôme de master en CM, avec deux suppléments de diplôme

1.3.2 -f Mention Relations Publiques et Multimédia CNMP

Parcours Communication Numérique et Management de Projet (CNMP)

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Communication Multimédia (CM)**.** Le Recrutement en S9 se fait par validation des crédits acquis.

La formation en Communication Numérique et Management de Projet de l'EMIT vise à former des professionnels polyvalents, compétents dans la gestion de projets numériques, le leadership d'équipes, et la compréhension des enjeux des médias numériques, en collaboration avec l'Université Bordeaux Montaigne en France.

1.3.2 -g Mention Relations Publiques et Multimédia RPCO

Parcours Relations Publiques et Communication Organisationnelle (RPCO)

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Relations Publiques et Communication Organisationnelle (RPCO)**.** Le Recrutement en S9 se fait par validation des crédits acquis.

Un parcours axé sur le développement de compétences stratégiques et opérationnelles. Les étudiants bénéficient d'une formation complète, couvrant la gestion de l'image de marque, la planification d'événements, la communication interne et externe, afin de les préparer à exceller dans le monde dynamique et exigeant des relations publiques.

1.4 Organigramme de l'EMIT

La structure hiérarchique au sein de l'EMIT se compose d'un conseil (scientifique ou d'établissement), une direction, un collège des enseignants, des chefs de mentions et des services présents au sein de l'école. Cette structure est représentée de manière générale sur la figure 1.1.

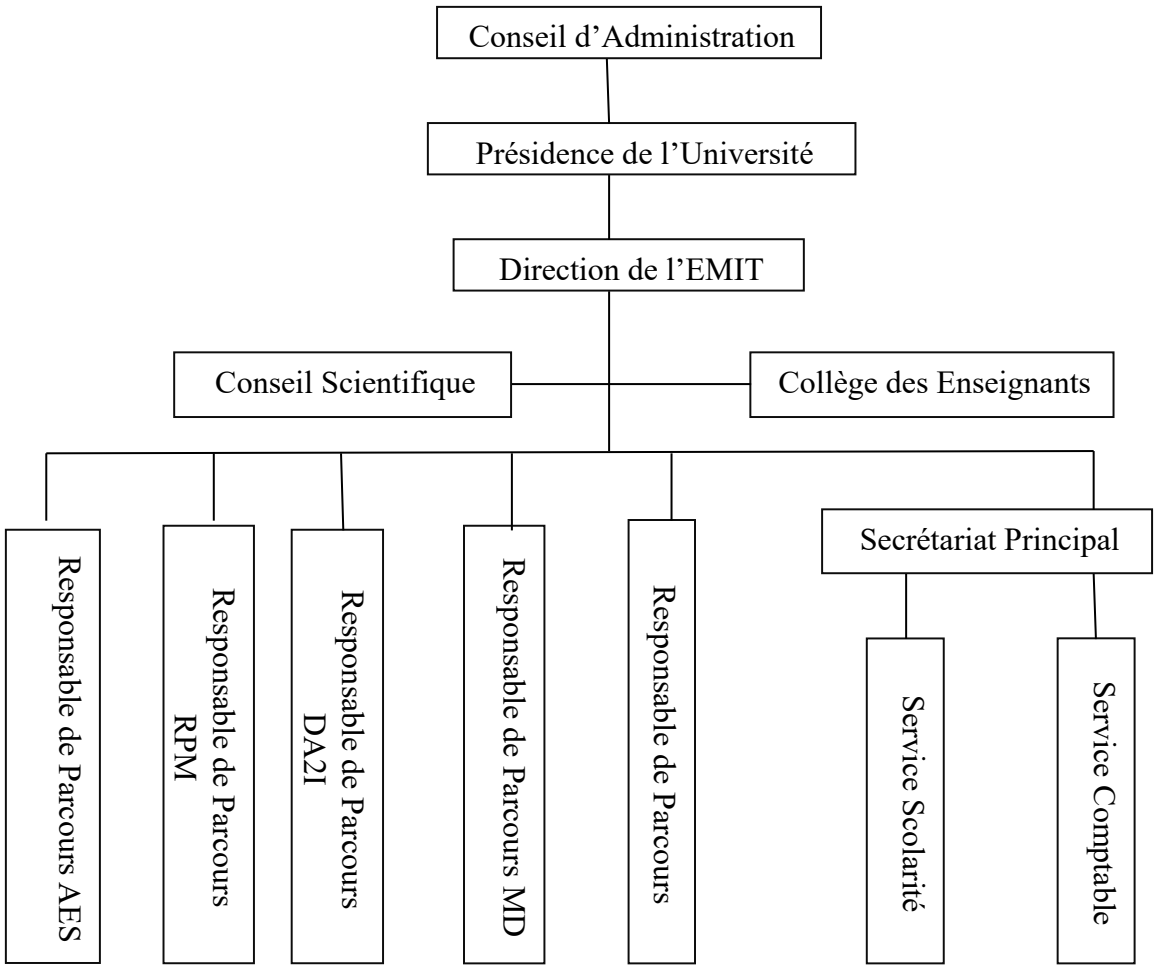

Figure 1.1 : Organigramme de l'EMIT

Source : Ecole de Management et d'Innovation Technologique

Chapitre 2 : PRESENTATION DE L'ENTREPRISE D'ACCUEIL

Dans le cadre de notre parcours académique et en vue de renforcer nos compétences pratiques, il nous a été donné l'opportunité d'intégrer une structure professionnelle dynamique et accueillant. Une immersion en milieu professionnel constitue une étape clé pour relier théorie et pratique. Avant de plonger dans le détail de nos missions, il est essentiel de présenter brièvement l'entreprise qui nous a accueillis.

Le stage s'est déroulé au sein de Merix, une maison de production audio dynamique spécialisée dans la création sonore, la production musicale, et le design audio pour divers formats multimédias. Merix s'inscrit dans une démarche artistique et technique qui place la qualité sonore au cœur de son identité.

2.1 À propos de Merix

Merix est une maison de production audio fondée en 2017 à Fianarantsoa par Tommy Gabrielo, passionné de musique, de technologie et de narration sonore. Née de l'envie de créer un espace d'expression et d'innovation artistique, Merix accompagne depuis sa création les artistes, entreprises et créateurs de contenu dans la réalisation de leurs projets sonores.

Dans un monde où la qualité du son est devenue un vecteur essentiel d'émotion et d'identité, Merix propose une approche exigeante et moderne de la production audio. Chaque projet est traité comme une œuvre unique, avec un soin particulier apporté à la direction artistique, au mixage, à l'enregistrement et à la post-production.

Chez Merix, nous croyons que le son est un langage universel capable de raconter, de rassembler et d'impacter profondément les publics.

La figure suivante représente le logo de MERIX :

Figure 2.1 Logo de MERIX

2.2 Nos Services

2.2.1 Production musicale

Merix accompagne les artistes dans toutes les étapes de création musicale : de la composition à l'enregistrement, en passant par le mixage et le mastering. Qu'il s'agisse de musique urbaine, traditionnelle, pop, ou expérimentale, l'équipe de Merix met à disposition des outils professionnels et un encadrement artistique de qualité.

Nos atouts :

- Studio d'enregistrement acoustiquement traité
- Équipements haut de gamme pour la prise de son et le traitement audio
- Réseau de musiciens et de techniciens expérimentés

2.2.2 Publicité sonore et identité audio

La signature sonore d'une marque est aujourd'hui aussi importante que son identité visuelle. Merix crée des jingles, habillages sonores et musiques publicitaires sur mesure, adaptés à l'image et aux valeurs de chaque client.

Avantages pour les entreprises :

- Meilleure mémorisation de la marque
- Valorisation des supports de communication
- Renforcement de l'impact émotionnel des campagnes

2.2.3 Podcasts et contenus narratifs

Merix accompagne les créateurs de podcasts et les institutions dans la production de contenus audio narratifs, depuis l'enregistrement jusqu'au montage final. Le storytelling audio est un puissant outil de communication que nous mettons au service des médias, ONG, artistes et entreprises.

2.3 Notre expérience et Notre Vision

Depuis sa création, Merix a collaboré avec plusieurs artistes indépendants, structures culturelles, agences publicitaires et réalisateurs audiovisuels. Chaque collaboration est pensée comme un partenariat créatif, avec une attention constante à la qualité, à l'écoute du client et à l'innovation.

Notre philosophie se résume ainsi : “Créer du son, c’est créer du lien.”

Au-delà de la technique, Merix est avant tout un espace de création, d’échange et d’expression artistique. Nous croyons que l’exigence sonore est un vecteur de professionnalisme et de différenciation sur tous les supports.

2.4 Contact

Pour toute demande d’information ou pour discuter de vos projets, n’hésitez pas à nous contacter :

Adresse : Lot2P86, Tambohobe, Fianarantsoa
Téléphones :

- 034 60 189 97
- 032 85 796 10

E-mail : merixcompany@gmail.com

Chapitre 3 : PRESENTATION DU PROJET

3.1 Origine du projet

Ce projet est né d'un besoin réel observé au sein de MERIX, confrontée à des difficultés dans la gestion quotidienne de ses activités : suivi des clients ; organisations des commandes ; gestion des services ; ainsi que le traitement des paiements. La gestion manuelle ou partiellement informatisée engendre souvent des pertes de temps, des erreurs de coordination et un manque de visibilité sur l'ensemble des opérations.

Bien que cette méthode ait permis une gestion basique des données, elle a montré rapidement ses limites face à la complexité et l'augmentation des tâches. En effet, l'usage d'Excel présente plusieurs inconvénients majeurs dans un contexte de gestion dynamique : absence d'automatisation, difficultés de partage de collaboratif en temps réal, faible sécurité des données, et manque d'historique fiable pour les suivis.

Face à cette situation, la création d'une application web s'est imposée comme une solution efficace pour centraliser les données, automatiser certaines tâches clés et améliorer la fluidité du travail entre les différentes intervenantes. Le projet est donc pour origine la volonté de proposer un outil simple, intuitif et accessible, capable de répondre aux besoins spécifiques de la maison de production audio. C'est-à-dire le fruit d'un besoin réel d'évolution technologique. Il s'inscrit dans une démarche d'amélioration continue du fonctionnement interne et de la qualité des services proposés, tout en facilitant le quotidien des responsables.

3.2 Contexte du projet

MERIX est une maison de production en pleine croissance, multipliant les projets artistiques, les collaborations et les prestations offertes à ses clients. Dans sa gestion quotidienne, elle s'appuyait jusqu'alors principalement sur des outils bureautiques classiques, tels que Microsoft Excel pour assurer le suivi de ses activités. Bien que fonctionnels à petite échelle, ces outils atteignent rapidement leurs limites dans un environnement dynamique et professionnel.

Le nombre croissant d'artistes, d'organisation, des collaborations, de projets audios et de sessions studio a rendu la coordination plus complexe. Dans ce contexte, il devient essentiel d'adopter un système capable de centraliser toutes ces informations, d'optimiser les procédures, et de permettre un accès rapide, sécurisé et fiable aux données de gestion.

3.3 Problématiques

L'utilisation d'Excel non connectés a montré de nombreuses limites dans un cadre professionnel exigeant. D'une part, l'absence de synchronisation en temps réel entre les différentes postes de travail provoquait des doublons, des conflits de planifications et des erreurs de saisie. D'autre part, la complexité croissante des projets ne permettait plus une visualisation claire et instantanée de l'état d'avancement ou des besoins spécifiques.

Le suivi des paiements, la production des bilans financiers fiables ou de générer des rapports détaillés sur les revenus par projet ou par période est une tâche très lourde pour Excel. La réservation des studios représentait également un point sensible : sans agenda partagé ou automatisé, des chevauchements de planning pouvaient survenir, occasionnant des tensions entre artistes ou ne mauvaise répartition des ressources matérielles.

A cela s'ajoutait une gestion limitée des données clients et des projets. Aucune base de données relationnelle n'était en place pour stocker de manière structurée les informations clés telles que les coordonnées des artistes, l'historiques des collaborations, ou les détails techniques des projets audios.

En résumé, le manque d'un système d'information intégré nuisait à la fluidité du travail, à la réactivité de l'Enterprise, à la qualité du service rendu, et à sa capacité à évaluer efficacement dans un marché concurrentiel. D'où la nécessité de mettre en place une application web de gestion sur mesure, adaptée aux spécificités du métier et capable de pallier durablement ces insuffisances.

3.4 Solution proposée

La solution proposée prend la forme d'une application web de gestion personnalisée, accessible depuis tout navigateur moderne. Elle permet une automatisation intelligente des processus internes de MERIX, tout en offrant une visualisation claire des données essentielles.

Parmi ses principales fonctionnalités :

- Un accueil interactif pour le suivi des recettes mensuelles et annuelles,
- Un système de gestion des données de clients en temps réel, et une base de données relationnelles bien centralisée des services, des commandes ainsi que les factures,
- Un module de facturation avec génération de téléchargement de documents formats PDF,

- Une interface de consultation des historiques des commandes,
- Un système d'authentification sécurisé.

Cette solution vise à remplacer les outils disparates actuellement utilisés par une plateforme unique, intégrée et évolutive, capable d'accompagner la croissance de l'activité et d'améliorer la qualité des services rendus.

3.5 Objectif du projet

L'objectif principal de ce projet est de développer une application web moderne adaptées aux besoins spécifiques d'une maison de production audio. Cette solution vise à centraliser et automatiser la gestion des activités courantes telles que le suivi des recettes annuelles et mensuelles, la facturation, la gestion des clients et services ainsi que l'historiques des commandes. Il s'agit de doter l'organisation d'un outil intuitif, fiable et évolutif, qui contribue à améliorer la productivité, la qualité des services offertes, et la fluidité des échanges internes.

En intégrant des technologies récentes de développements web et une base de données relationnelle robuste, cette application devra offrir une interface conviviale et sécurisée, tout répondant aux exigences professionnelles tout en simplifiant les tâches quotidiennes.

3.6 Conduite du projet

La réalisation de ce projet s'est appuyée sur une démarche structurée, itérative et orientée vers les besoins réels de l'entreprise. Elle a débuté par une phase d'analyse approfondie des pratiques existantes et des attentes fonctionnelles des utilisateurs. Cette étape a permis de collecter les exigences essentielles à la conception d'une solution adaptée.

Ensuite, la modélisation du système a été menée selon méthode Merise II, afin de garantir une structuration rigoureuse des données et des traitements. Le développement a été découpé en plusieurs étapes : analyse fonctionnelle, modélisation et conception, création de l'interface utilisateur, mise en place des fonctionnalités clés, puis phases de test et d'ajustement en fonction des retours.

Une attention particulière a été accordée à l'ergonomie de l'application, à la clarté de la navigation et à la fiabilité des interactions avec la base de données. Le projet a été conduit de manière agile, en intégrant régulièrement les retours de l'Enterprise d'accueil afin de coller au plus près de se réalités opérationnelles.

Partie II : Analyse et Conception du projet

Chapitre 4 : METHODES, NOTATIONS UTILISEES

4.1 Méthodes

Pour mener à bien ce projet et d'assurer une démarche rigoureuse tout au long du développement de l'application de gestion destinée à une maison de production audio, une approche méthodique a été indispensable pour être un cadre de référence. Le choix s'est porté sur la méthode Merise, plus précisément Merise II, une évolution de la méthode Merise traditionnelle, reconnue pour sa clarté et sa structuration précise dans les projets de conception de systèmes d'informations.

4.1.1 Comparaison avec UML

Une autre alternative couramment utilisée dans les projets orientés objet et UML (Unified Modeling Language). UML est très adapté pour modéliser les systèmes orientés objets, en particulier dans des environnements de programmation comme Java, C++ ou Python. Il excelle lorsqu'il s'agit de concevoir des classes, des interactions objets ou des diagrammes de cas d'utilisation.

Cependant, le projet ici est fortement centré sur les données (clients, projets, paiements, utilisateurs, tâches, etc.), et moins sur une logique objet complexe. De plus, UML propose une très grande variété de diagrammes (plus d'une dizaine), parfois redondants ou trop détaillés pour un projet de cette envergure.

A l'inverse de Merise II se concentre sur les aspects les plus critiques pour un système de gestion : les données, les traitements et les flux d'information, avec une structure claire en trois niveaux (conceptuel, organisationnel, physique). Cela le rend particulièrement adapté pour les bases de données relationnelles comme MySQL ou PostgreSQL.

4.1.2 Avantages spécifiques de Merise II dans ce projet

4.1.3 -a Séparation rigoureuse des préoccupations :

Grâce à sa structuration en niveaux (conceptuel, logique, physique), Merise II permet d'éviter toute confusion entre la conception métier et la mise en œuvre technique.

4.1.3 -b Documentation claire et progressive :

Chaque livrable (MCD, MLD, MPD, MCT) constitue une brique indépendante, facile à expliquer et à justifier devant un jury ou un client.

4.1.3 -c Fiabilité des données :

La méthode impose une modélisation rigoureuse qui limite les anomalies (redondances, oublis, incohérences), un point critique dans une application de gestion.

4.1.3 -d Adaptation au contexte pédagogique :

En somme, après une analyse comparative des différentes approches de modélisation, il apparaît que Merise II s'impose comme la méthode la plus pertinente pour ce projet de développement d'une application web de gestion de services pour MERIX. Sa capacité à bien distinguer les niveaux conceptuels, organisationnels et techniques, ainsi que sa rigueur dans la modélisation des données et des braiments, permet une meilleure anticipation des besoins évolutifs du système.

Contrairement à certaines méthodes agiles ou centrées uniquement sur l'implémentation rapide, Merise II offre une vision globale et stable, particulièrement utile pour un projet dont la cohérence des données et la pérennité des processus sont prioritaires. Elle permet aussi une documentation claire, facilitant le passage de relais, la maintenance et les évolutions futures de l'application.

4.2 Notations utilisées

Dans toutes démarche rigoureuse de conception d'un système d'information, le choix des notations joue un rôle déterminant. Il ne s'agit pas seulement de représenter des données ou des processus, mais de structurer la pensée, standardiser la communication entre les acteurs du projet, et assurer une continuité fluide entre les phases d'analyse, de conception et de réalisation. Dans le cadre de notre projet, nous avons adopté la méthode Merise II, qui offre une panoplie de notations adaptées à chaque niveau d'abstraction : conceptuel, logique et physique.

4.2.1 La modélisation conceptuelle

La modélisation conceptuelle est une étape fondamentale dans le processus de développement d'une application. Elle permet de représenter de manière abstraite et indépendante des contraintes techniques, les besoins réels du système à concevoir. L'objectif est de structurer l'information en mettant en évidence les entités, leurs attributs, et les relations qui les lient.

Dans ce projet de gestion pour une maison de production audio, l'étape conceptuelle a permis de transformer les besoins exprimés en un schéma clair et logique, compréhensible par tous les acteurs du projet, qu'ils soient techniques ou non.

4.2.1 -a Le Modèle Conceptuel des Données (MCD)

Le MCD représente les données des manières abstraites, sous forme d'entités (clients, artistes, contrats, productions, etc.) et de relations (possède, enregistre, produit…). Chaque entité est décrite avec ses attributs caractéristiques.

Par exemple :

- **Client :** nom, prénom, contact, adresse
- **Artiste :** nom de scène, genre musical, nationalité
- **Production :** titre, date, type

Le MCD permet une vision claire et indépendante de la structure des données, sans se préoccuper de la manière dont elles seront stockées.

4.2.1 -b Le Modèle Conceptuel des traitements (MCT)

Le MCT décrit les processus métier, c'est-à-dire les traitements que le système doit effectuer. Chaque traitement est défini indépendamment de sa future implémentation.

Par exemple :

- Enregistrement d'un nouveau client
- Gestion des services de production
- Suivi des activités et commandes

Le MCT structure les opérations à effectuer en lien avec les données du MCD. Il décrit les activités découlant des échanges entre le système étudié et le monde extérieur. Dont les concepts de base du MCT sont : Acteur, Evénement, Opération, Réglé d'émission, la synchronisation.

4.2.1 -c Modèle Conceptuel de Flux (MCF)

Le MCF aussi appelé les diagrammes de flux d'informations ou modèle de flux conceptuel. Il modélise les échanges d'informations entre les acteurs externes et le système. Représenter les flux d'informations consiste à analyser les échanges d'informations au sein du système d'information d'une organisation (entreprise, administration ou association) et avec d'autres systèmes d'information.

4.2.2 La modélisation logique

Le modèle logique occupe une place centrale dans la méthode Merise, située entre la phase conceptuelle (indépendante de toute technologie) et la phase physique (liée à l'implémentation technique réelle). Il a pour objectif de traduire les modèles conceptuels en une structure formelle et cohérente qui pourra être facilement transformée en une base de données ou un système informatique, tout en restant indépendant du SGBD (Système de Gestion de Base de Données) ou de la technologie utilisée.

Objectifs du modèle logique :

- Structurer l'information de manière cohérente et non redondante.
- Définir clairement les relations entre les données (clés primaires, étrangères).
- Préparer l'implémentation tout en restant indépendant des choix techniques concrets (ex : MySQL, PostgreSQL, Oracle).
- Respecter les règles de gestion identifiées dans le modèle conceptuel.

Les deux grandes catégories du modèle logique dans Merise II sont : MLD (Modèle Logique de Données) et MLT (Modèle logique de Traitement).

4.2.2 - a Modèle Logique de Données (MLD)

Le MLD traduit le MCD dans un langage relationnel, préparation à l'implémentation dans SGBDR (ex : MySQL, PostgreSQL). Autrement dit une étape intermédiaire pour passer du modèle E/A (MCD), qui est un modèle sémantique, vers une représentation physique des données : fichiers, SGBD hiérarchique, SGBD réseau, SGBD relationnel.

Les règles de transformation appliquées :

- Chaque entité devient une table
- Chaque attribut devient une colonne
- Les associations selon leur cardinalité deviennent soit des relations directes, soit des tables d'associations avec des clés étrangères.

4.2.2 - b Modèle Logique de Traitement (MLT)

Le MLT formalise les traitements selon une logique de programmation (pseudo-code) :

- Il décrit les règles métier, les calculs à effectuer, les boucles ou conditions.
- Il est souvent rédigé sous forme de diagramme de flux ou de pseudo-algorithmes, proches du code source final.

4.2.3 Modélisation organisationnelle

Le modèle organisationnel constitue une étape intermédiaire entre la modélisation logique (structurée mais encore abstraite) et la modélisation physique (prête pour l'implémentation technique). Il vise à intégrer les réalités concrètes de l'organisation humaine et matérielle dans la conception du système d'information.

Autrement dit, à ce stade, on ne se contente plus de décrire les données et les traitements de manière logique. On cherche à répondre à des questions comme :

- Qui effectue tel traitement ?
- Où est-il réalisé ?
- A quel moment ou selon quel déclencheur ?
- Avec quels moyens techniques (documents, outils, logiciels) ?
- Quels services sont responsables de chaque information ?

Ce niveau est crucial pour adapter le système d'information aux pratiques et contraintes réelles de l'entreprise.

Objectifs du modèle organisationnel :

- Identifier les acteurs humains et systèmes impliqués dans les processus.
- Associer les traitements et les données aux services, postes ou rôles réels.
- Prendre en compte les contraintes de temps, de lieu et de support (papier, informatique, mobile…).
- Préparer la modélisation physique avec une vision réaliste du fonctionnement opérationnel.

Deux sous-modèles clés composent le modèle organisationnel : MOT (Modèle Organisationnel des Traitements) et MOD (Modèle Organisationnel des Données).

4.2.3 -a Modèle Organisationnel des Traitements (MOT)

Le MOT spécifie en détails le contenu de chaque opération conceptuelle. Il décrit la matière dont ces fonctions sont matériellement assurées.

C'est de traduire les processus métiers dans l'organisation réelle.

Responsable de traitement : personne ou service qui exécute un processus.

Fréquence : combien de fois un traitement est réalisé.

Déclencheur : ce qui provoque un traitement (ex : réception d'une commande).

Durée : temps estimé du traitement.

4.2.3 -b Modèle Organisationnel des Données (MOD)

On commence à introduire les contraintes de l'environnement réel (accès, responsabilités…).

Responsable d'entité : qui gère une entité donnée.

Lieu de traitement : où les données sont manipulées (poste, département, etc…).

Droits d'accès : précise qui peut lire, écrire ou supprimer des données.

4.2.4 Modélisation physique

Le modèle physique est la dernière étape de la modélisation des données dans la méthode Merise II. Il correspond à la traduction concrète et technique des modèles précédents (conceptuels et logiques) dans un environnement réel, c'est-à-dire dans un Système de Gestion de Base de Données (SGBD) spécifique comme MySQL, Oracle, PostgreSQL, etc.

Il répond à la question : « Comment stocker, organiser et optimiser les données dans une base de données réelle ? ».

Objectifs du modèle physique :

- Transformer le Modèle Logique de Données (MLD) en une structure exploitable techniquement ;
- Définir les types de données adaptés (VARCHAR, INT, DATE, etc…) ;
- Définir les clés primaires, clés étrangères et contraintes d'intégrité ;
- Créer des index pour optimiser les performances ;
- Prévoir des règles de gestion de l'espace mémoire, de sécurité ou de réplication si besoin.

Les contenus d'un MPD (Modèle physique de Données) sont :

Les tables physiques avec leur structure complète ;

- Les types exacts de chaque champ selon le SGBD ;
- Les contraintes : NOT NULL, UNIQUE, DEFAULT, CHECK, etc… ;
- Les clés primaires et étrangères ;

- Les index pour accélérer les recherches ;
- Parfois, les vues, procédures stockées, triggers, etc….

Le Modèle Physique des Données (MPD) est l'étape où la conception théorique devient une réalité technique. Il permet à la base de données de fonctionner dans un contexte informatique réel, en garantissant la cohérence, la performance et la fiabilité des données.

En somme, l'utilisation conjointe de ces notations assure un développement rigoureux, traçable et maîtrisé du projet. Ce découpage méthodologique rend possible une collaboration fluide entre les parties prenantes, une validation progressive à chaque étape, et garantit la qualité finale de l'application développée. Le choix de Merise II et de ses notations s'impose donc comme un pilier structurant de ce projet, justifiant pleinement son adoption dans un contexte professionnel et pédagogique.

Chapitre 5 : ANALYSE DU PROJET ET ETUDE DE L'EXISTANT

L'analyse du projet constitue une étape cruciale pour comprendre l'environnement dans lequel l'application sera développée et déployée. Elle permet d'identifier les besoins réels, les attentes des futurs utilisateurs. Ce chapitre est structuré autour de deux axes : une analyse fonctionnelle de l'environnement de travail actuel et une étude approfondie des outils existants, notamment les méthodes traditionnelles utilisées au sein de la maison de production audio.

La réussite d'un projet informatique repose en grande partie sur la rigueur de son analyse préalable. Avant de se lancer dans la conception technique et le développement d'une application, il est impératif de comprendre en profondeur le contexte dans lequel celle-ci sera utilisée.

5.1 Présentation de la structure

La maison de production concernée par ce projet est une entité spécialisée dans la création, l'enregistrement, le mixage, prise de son, mastering, instrumental et la diffusion de contenus audio, tels que les chansons, les spots publicitaires, les podcasts ou encore les voix-off. Elle travaille avec des artistes, des entreprises, des médias, et des particuliers.

Elle est composée de plusieurs services clés :

- Studio d'enregistrement,
- Service client,
- Direction administrative,
- Equipe technique et artistique.

Le fonctionnement repose principalement sur des interactions fréquentes entre ces pôles pour la planification des sessions, la gestion des fichiers produits, le suivi des commandes, et la facturation.

5.2 Description du système actuel

Actuellement, la maison de production gère ses activités quotidiennes telles que les gestions informations clients, les données des services ainsi que les commandes, la facturation à l'aide d'un outils bureautique classiques, principalement Microsoft Excel.

Limites identifiées

Répétition et redondance des données : les informations sont saisies plusieurs fois dans des différents documents.

Risque de perte de données : aucun système de sauvegarde centralisée n'est mis en place.

Manque de traçabilité : il est difficile de retrouver l'historique complet d'un client ou d'un service.

Temps de traitement long : chaque tâche nécessite un suivi manuel chronophage.

Absence de collaboration en temps réel : aucun système ne permet aux membres d'une équipe de travailler simultanément sur un projet ou un fichier.

Risque d'erreur humain : durant les saisies des nombreuses possibilités d'erreur.

5.3 Analyse des besoins fonctionnels

L'analyse des besoins repose sur des entretiens semi-directifs menés avec les utilisateurs clés de la structure. Les attentes fonctionnelles ont été traduites en fonctionnalités prioritaires.

5.3.1 Besoins principaux

- Gestion des fichiers clients (coordonnées, services associés).
- Suivi des projets et des services artistiques.
- Génération en format PDF des factures.
- Téléversement et archivage sécurisé des fichiers produits.
- Tableau de bord avec les recettes annuelles et mensuels.

5.3.2 Besoins non fonctionnels

- Application responsive (accessible depuis ordinateur et mobile).
- Interface simple et ergonomique (adaptée aux utilisateurs non techniques).
- Système sécurisé et fiable, avec des sauvegardes régulières.

5.4 Analyse des outils concurrents

Plusieurs solutions logicielles ont été brièvement examinées dans le cadre de cette étude. Cependant, les outils existants sur le marché sont souvent trop génériques ou coûteux,

sans prise en compte des spécificités propres à une maison de production audio locale. La majorité des logiciels propose des fonctionnalités standards, mais :

- Leur coût d'abonnement est élevé, souvent en devise étrangère.
- Ils sont parfois inaccessibles hors connexion ou nécessitent une bonne connexion internet.
- Ils ne sont pas adaptés au contexte local, notamment pour les formats de facturation et les types de services proposés.
- Leur interface est souvent en anglais ou surchargée de modules inutiles pour une petite structure.

De plus, ils ne sont pas nécessairement adaptés au contexte de travail de l'établissement, qui nécessite une interface simple, en français, avec des fonctionnalités sur mesure. Cela confirme la nécessité de concevoir une solution personnalisée répondant aux besoins spécifiques de la maison de production.

Objectifs du projet

A partir de cette analyse, les objectifs du projet peuvent être formulés ainsi :

- Remplacer les outils bureautiques traditionnels par une application web centralisée.
- Automatiser les processus clés.
- Amélioration de la traçabilité des projets et la productivité des équipes.
- Offrir une meilleure expérience utilisateur grâce à une gestion rapide et claire.
- Réduire les erreurs humaines et les pertes de données.

En conclusion, l'analyse de l'existant met en évidence des failles importantes dans le système de gestion. Bien que fonctionnel pour une activité de petite envergure, ce mode opératoire atteint aujourd'hui ses limites face à l'évolution des besoins.

La mise en place d'une application web moderne permettra non seulement de professionnaliser la gestion des services, mais aussi d'optimiser le temps, la coordination, la fiabilité et la qualité de service. Cette transition s'inscrit pleinement dans une dynamique de digitalisation adaptée aux réalités et aux ressources de la structure.

Ainsi, la modernisation du système d'information ne constitue pas une simple option, mais une nécessité stratégique pour accompagner le développement de la maison de production.

Chapitre 6 : CONCEPTION DU PROJET

La phase de conception est cruciale dans tout projet informatique. Elle représente la passerelle entre l'analyse approfondie de l'existant et la mise en œuvre technique. L'objectif de cette étape est de traduire les besoins fonctionnels et non fonctionnels identifiés en une architecture claire, stable et évolutive. Cela passe par la modélisation des données, des traitements, des échanges et des interactions entre les différents composants du système.

Objectifs de la conception :

- Définir de manière structurée l'architecture générale du futur système.
- Déterminer les composants fonctionnels essentiels du projet.
- Eviter les redondances et anticiper les conflits logiques.
- Poser les bases pour une implémentation cohérente, alignée avec les exigences.

6.1 Dictionnaire des données

Tableau 6.1 Dictionnaire de donnée

Nom	Description	Type	Taille	Format	Contraintes
Id_Cli	Identification clients	Entier	3		Clé primaire
Nom_Cli	Nom clients	Chaîne de caractères	100		Obligatoire
Telephon_Cli	Téléphone clients	Chaîne de caractères	16	+ [code pays] [numéro nationale]	Obligatoire
Adresse_Cli	Adresse clients	Chaîne de caractères	100		Facultatif
Email_Cli	Email clients	Chaîne de caractères	254		Facultatif
Num_services	Numéro du service	Entier	3		Clé primaire /Clé étrangère
Titre_services	Titre du service	Chaîne de caractères	100		Obligatoire

Types_services	Type du service	Chaîne de caractères	100		Obligatoire
Unite_services	Unité de service	Chaine de caractère	100		Obligatoire
Id_Commandes	Identification commande	Entier	3		Clé primaire /Clé étrangère
Date_Commandes	Date du jour de commande	Date		aaaa/jj/mm	Obligatoire
Date_realisation	Date de réalisation de la commande	Date		aaaa/jj/mm	Obligatoire
Date_livraison	Date de livraison de la commande	Date		Aaaa/jj/mm	Obligatoire
Id_Clients	Identification client	Entier	3		Clé étrangère
Quantite_services	Quantité de services	Entier	3		Obligatoire
Prix_unitaire	Prix unitaire d'un service	Décimal	15,2		Obligatoire
Total	Prix total d'une ligne de commande	Décimal	15,2		Stored Generated (automatiquement calculée)
Num_Utilisateurs	Identifiant utilisateur	Entier	3		Clé primaire, clé étrangère
Nom_Utilisateurs	Nom de l'utilisateur	Chaîne de caractère	100		Obligatoire
Email_Utilisateurs	Adresse e-mail de l'utilisateur	Chaîne de caractère	254		Obligatoire
Mot_de_passe	Mot de passe de l'utilisateur	Chaîne de caractère	64		Obligatoire
id	Numéro du connexion	Entier	3		Clé primaire

date_connexion	Date et heure de connexion	Date et heure		yyyy-mm-dd hh : mm : ss	Obligatoire
ip	Adresse IP de l'appareil lors de la connexion	Chaîne de caractère	100		Obligatoire
device	Représente la propriété de l'appareil de connexion	Chaîne de caractère	100		Obligatoire
location	Localisation de l'appareil de connexion	Chaîne de caractère	100		Obligatoire
success	Représente le succès du connexion	Booléen	1		Obligatoire
Id_Studio	Numéro des données sur le studio	Entier	3		Clé primaire
Adresse	Adresse de la maison de production audio	Chaîne de caractère	100		Obligatoire
Ville	Ville d'emplacement du studio	Chaîne de caractère	100		Obligatoire
Code_Postale	Code postale de la ville d'emplacement	Entier	3		Obligatoire
Contact	Contact du studio	Chaîne de caractère	100		Obligatoire
Nif	Numéro	Entier	20		Obligatoire

	d'Identification Fiscale	long			
Stat	Numéro Statistique	Entier long	20		Obligatoire
Responsable	Nom du responsable du studio	Chaîne de caractère	100		Obligatoire

Ce dictionnaire de données représente une documentation qui centralise toutes les informations détaillées sur les données manipulées dans le SI.

6.2 Dictionnaire de traitement

Ce dictionnaire de traitement représente un document structuré qui décrit de manière précise tous les traitements du système d'information étudiée, en complément du dictionnaire de données.

Tableau 6.2 Dictionnaire de Traitement

Nom du traitement	Type	Description	Entrée	Sorties	Contraintes	Commentaires
Création d'un nouvel enregistrement	Interactif	Permet d'ajouter un nouvel élément (client, services, commandes et lignes de commandes) via un formulaire	Champs saisis ou sélectionner par l'utilisateur	Enregistrement ajouté à la table correspondante	Champs obligatoires remplis, données valides	Ne permet pas l'ajout des doublons, vérification des formats des données ajoutées
Affichage des données	Automatique	Permet d'afficher automatique	Requête utilisateur	Données afficher dynamique	Affichage ne descenda	Données afficher avec des

		ment les données ajoutées dans la base sous forme de liste		ment dans la liste	nt des données en fonctions de leur identifiant	boutons de modifications et de suppression
Mise à jour des données	Interactif	Modifie les informations existantes dans la base de données	Identifiant + nouvel données	Données mises à jour	Validation des données	Formats des données requises
Suppression des données	Interactif	Supprime un enregistrement de la base	Identifiant	Enregistrement supprimé	Message de confirmation avant suppression	Suppression logique possible
Filtrage dynamique	Interactif	Recherche et filtrage des données en temps réel des données afficher selon les critères	Critères saisis	Liste filtrée ou recette filtrée	Respect de la syntaxe de recherche	N'est pas sensible à la casse
Authentification utilisateur	Interactif	Vérifie les identifiants pour autoriser l'accès à	Email et mot de passe	Accès accordé ou refusé	Mot de passe peut être afficher ou	Tentatives limitées

		l'application			masquer selon le choix de l'utilisateur	
Changement de mot de passe	Interactif	Permet à l'utilisateur de modifier son mot de passe	Email lors de l'authentification, nouveau mot de passe et confirmation	Nouveau mot de passe enregistré	Respect des règles de sécurité (longueur, complexité)	Confirmation requise
Exportation en PDF	Interactif	Génère un PDF à partir des données	Données à exporter	Fichier PDF téléchargé	Mise en page définie	Logo et un espace signature inclus
Sécurité	Automatique	Enregistre automatiquement les données lors d'une authentification de l'utilisateur	Nom de l'utilisateur, date et heure de connexion, adresse IP, localisation, appareil et succès	Données enregistrées à la base de données et affichées dans la partie sécurité du paramètre	Les connexions lors des blocages cause des tentatives limitées ne sont pas enregistrées	
Historique	Automati	Affichage	Les	Listes des	Les	Les

	que	des historiques des commandes dans l'application	informations sur les commandes	commandes afficher selon les critères	listes sont affichées en fonctions de leur mois	données affichées par défaut est spécifié en commande du
Notification						

6.3 Modèle de flux conceptuel (MFC)

Le MFC décrit les flux d'informations échangés entre le système d'information et ses acteurs externes. Pour optimiser la présentation du MFC dans la partie conception du projet, nous avons choisi de présenter graphiquement seulement la gestion clients.

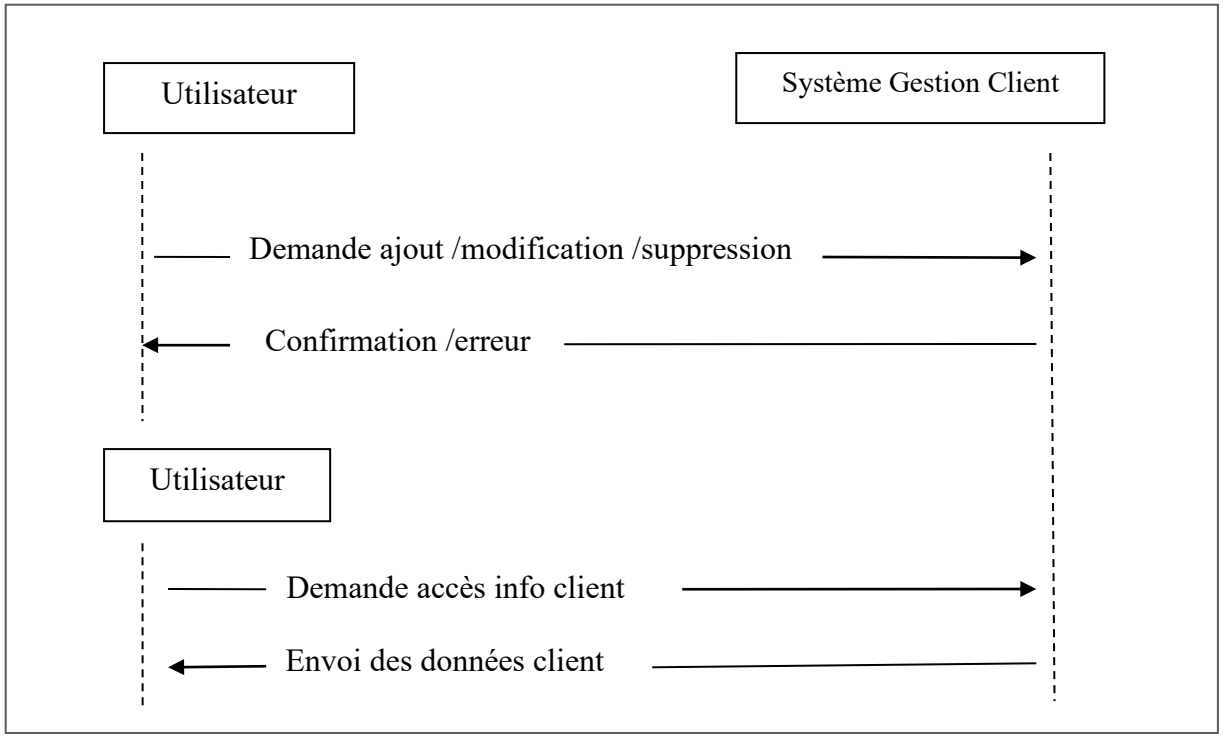

Figure 6.1 : Modèle de Flux Conceptuel

6.4 Modèle conceptuel de traitement (MCT)

Le MCT traite dynamiquement le SI, les opérations qui sont réalisées en fonction des événements. Il représente de façon schématique l'activité du SI sans faire référence à des choix organisationnels ou des moyens d'exécution. Pour optimiser, le MCT représenté ci-dessous est juste le côté ajout client.

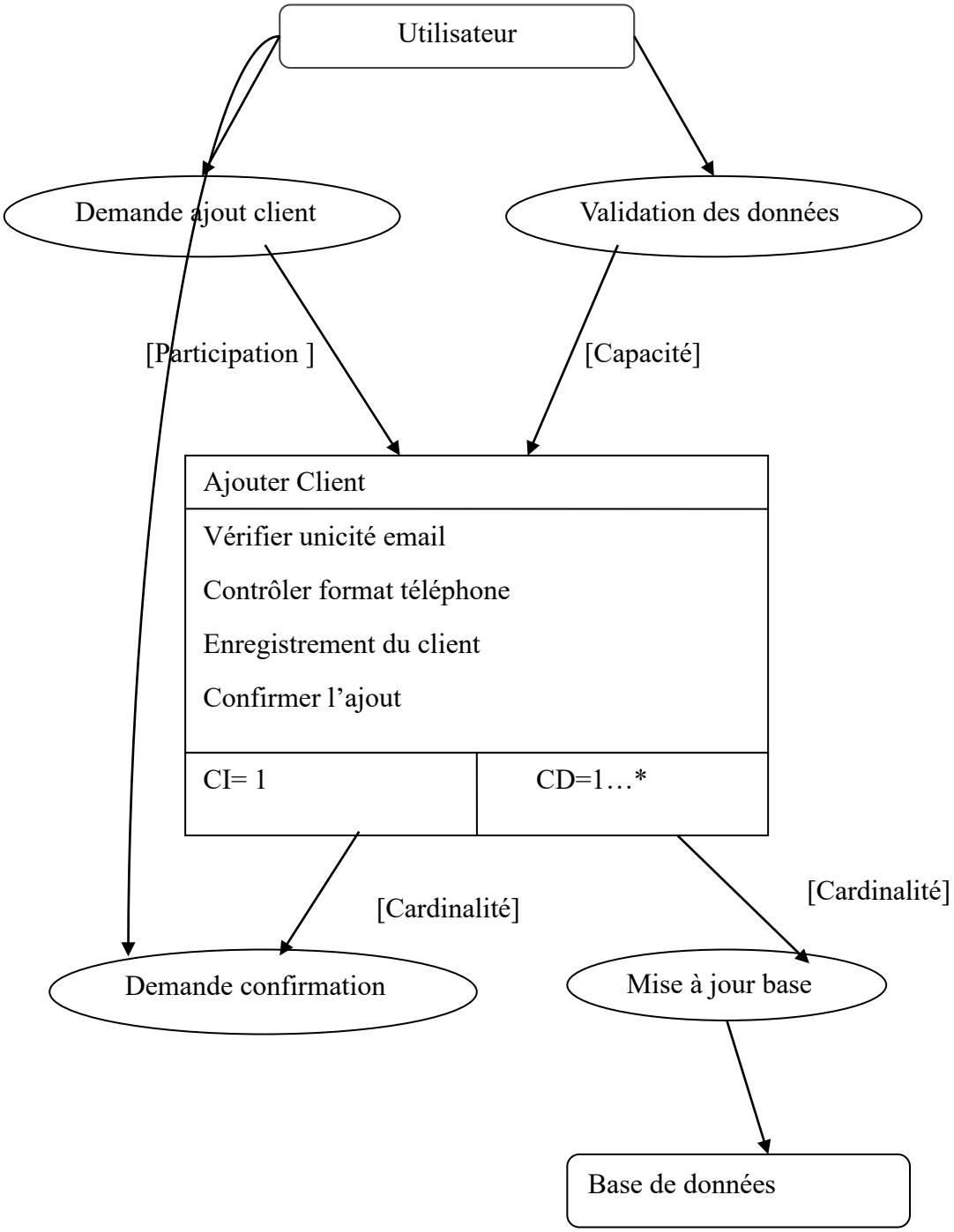

Figure 6.2 : Modèle conceptuel de traitement

6.5 Modèle conceptuel des traitements analytiques (MCTA)

Le MCTA est une représentation synthétique des traitements qui met en évidence les réactions du système aux événements et l'interaction avec les données. Pour optimiser, la présentation sera selon l'ajout client.

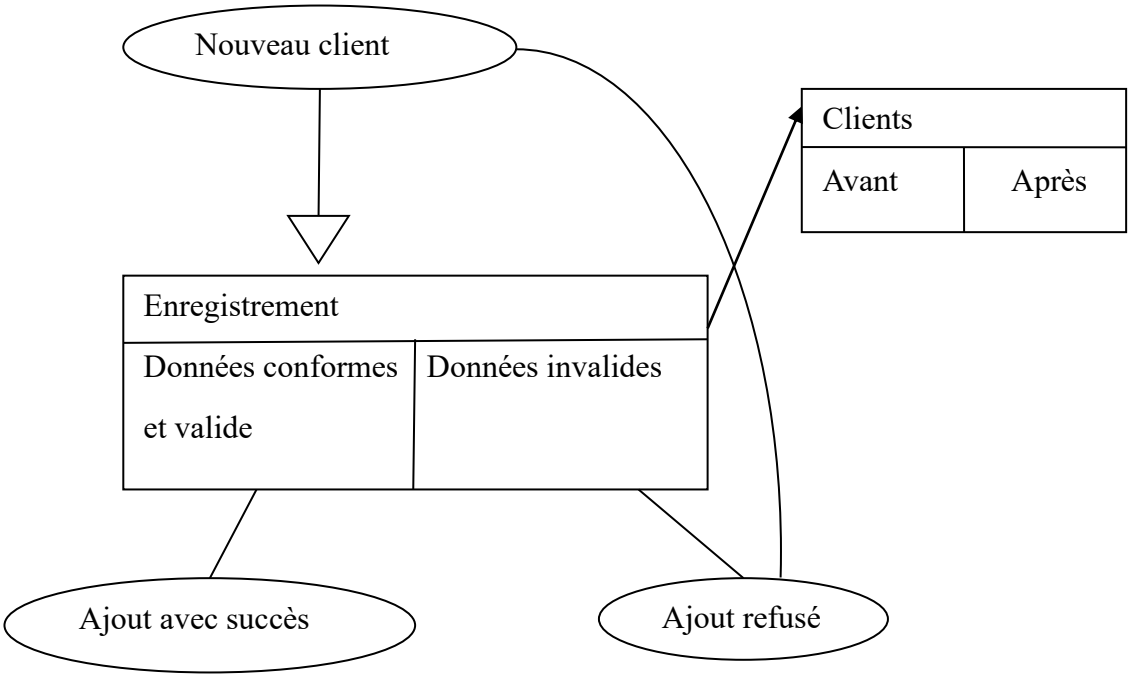

Figure 6.3 : Modèle conceptuel des traitement analytiques

6.6 Modèle organisationnel de traitements

Le MOT spécifie en détails le contenu de chaque opération conceptuelle. Pour optimiser, une présentation graphique de la gestion clients simplifiée.

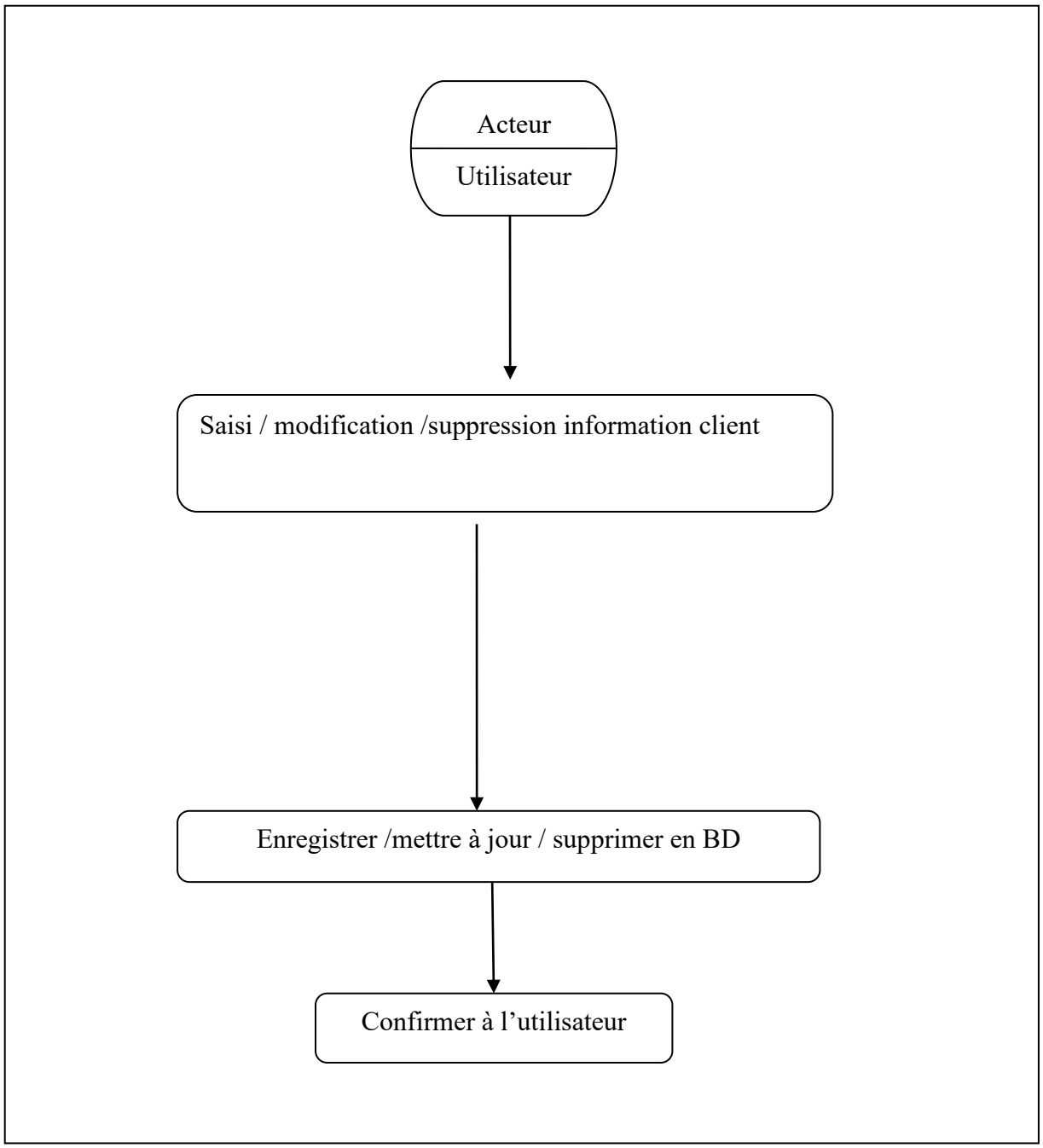

Figure 6.4 : Modèle Organisationnel de Traitements

6.7 Modèle organisationnel de traitements analytiques

Le MOTA est une version de présentation de MOT plus spécifique et analytique.

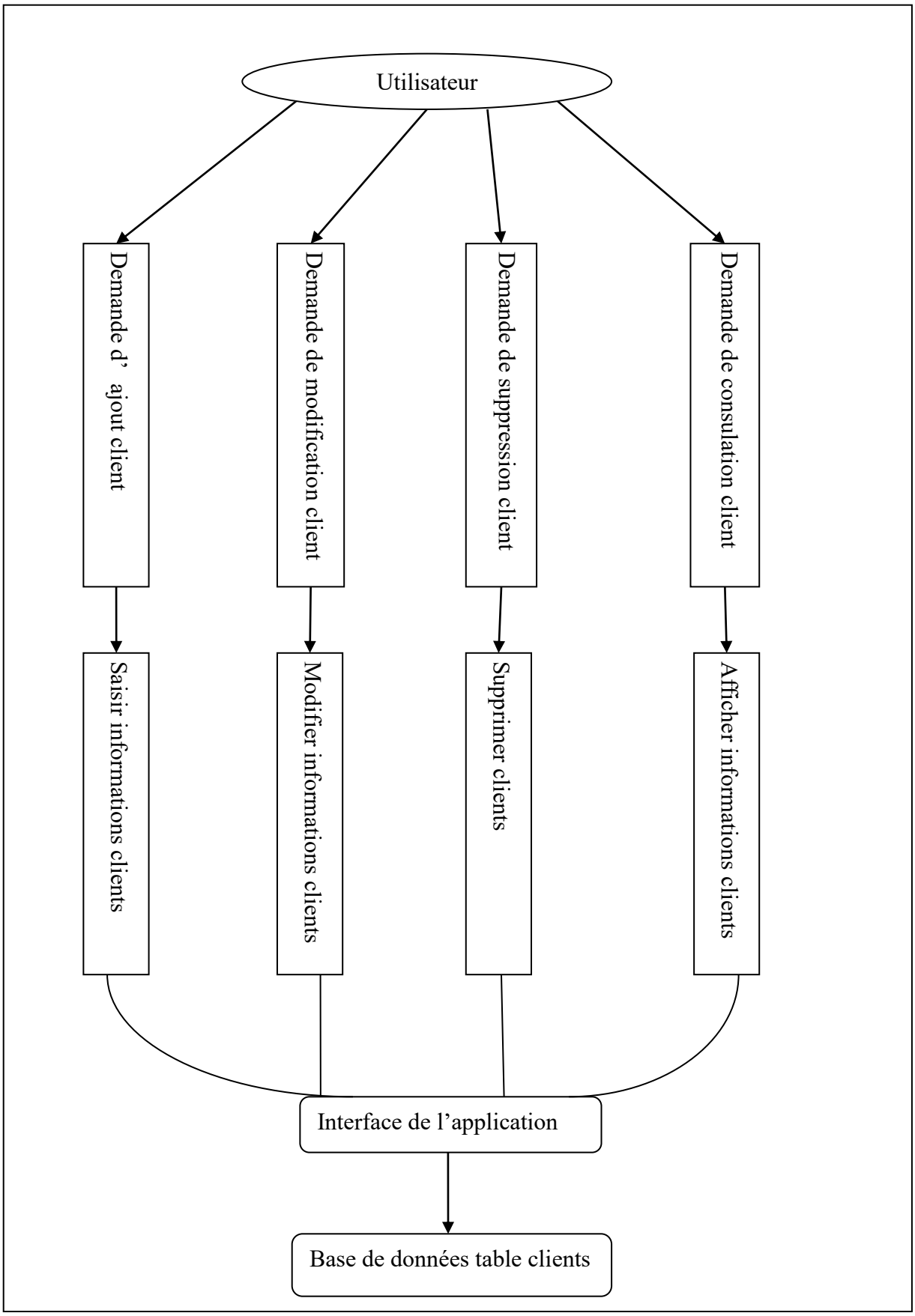

Figure 6.5 : Modèle organisationnel de traitement analytique

6.8 Règle de gestion

RG1 : Un client peut effectuer une ou plusieurs commandes. Une commande est passée par un seul client.

RG2 : Une commande est passée à une date donnée et peut avoir des dates de réalisation et de livraison.

RG3 : Une commande contient au moins un service. Plusieurs services peuvent être ajoutés à une même commande.

RG4 : Un service peut être demandé dans plusieurs commandes différentes. Un seul service se trouve dans une ligne de commandes

RG5 : Le prix d'un service est négocié pour chaque commande.

RG6 : Chaque ligne de commande précise la quantité de service demandée et le prix négocié.

RG7 : Le montant d'une commande est calculé à partir des lignes de commande (quantité × prix négocié).

RG8 : Un utilisateur doit avoir un compte d'authentification pour accéder aux contenues de l'application, dont il peut avoir connexion mais une connexion est effectuée par un seul utilisateur.

RG9 : Chaque connexion est enregistrée pour la sécurité de l'application, et l'utilisateur a des tentatives limitées pour fluidité de la sécurité.

Ces règles de gestion sont les contraintes et conditions définies par l'organisation qui régissent : le fonctionnement des processus, la manipulation des données, et les décisions à prendre dans un SI.

6.9 Modèle conceptuel de donnée (MCD)

Le MCD est la première étape dans la modélisation des données selon la méthode Merise. Il permet de représenter, de manière graphique et indépendante de toute technologie, l'organisation logique de données du système développer.

Dans le cadre de notre projet le MCD se décompose en trois volets fonctionnels :

Gestion générale : inclut les informations globales et nécessaires pour la gestion des services dans la maison de production audio, tels que les données clients, services, commandes et lignes de commandes.

Gestion sécurité : inclut les informations de l'utilisateur et ses connexions.

Les données personnelles : inclut la table qui contient les informations sur le studio.

6.9.1 Gestion générale

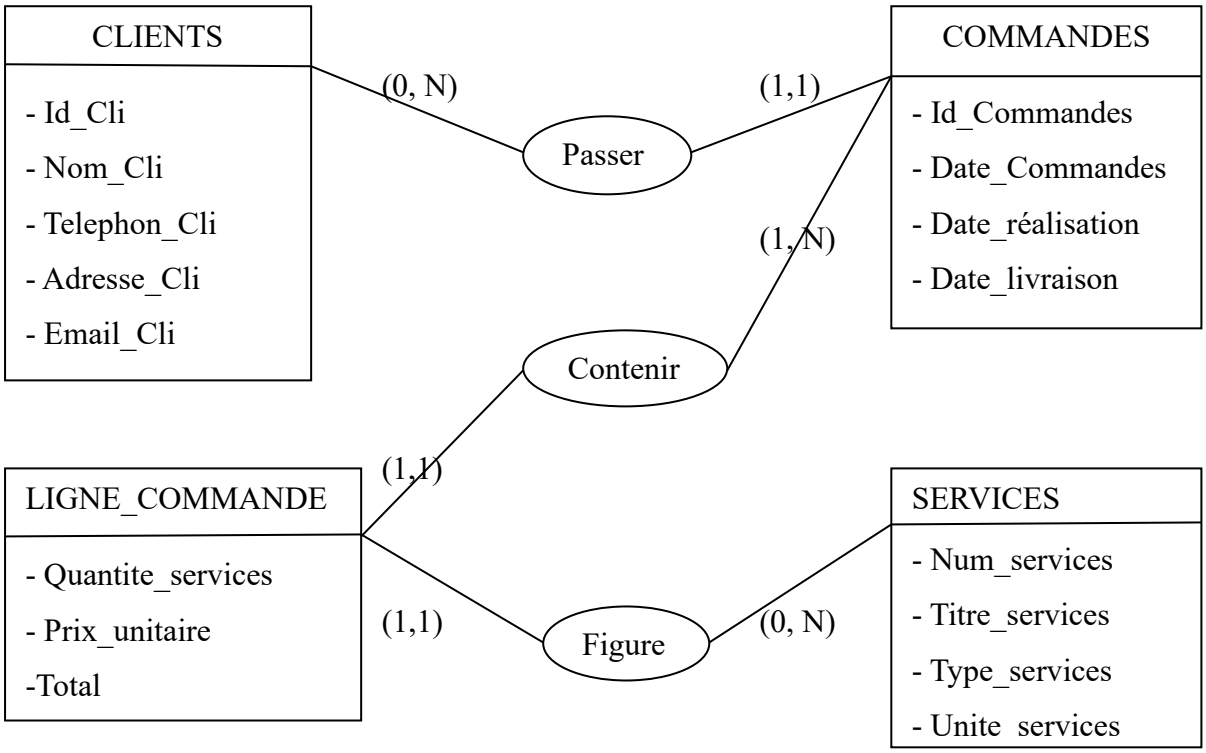

Figure 6.6 : Modèle conceptuel de données générale

6.9.2 Gestion sécurité

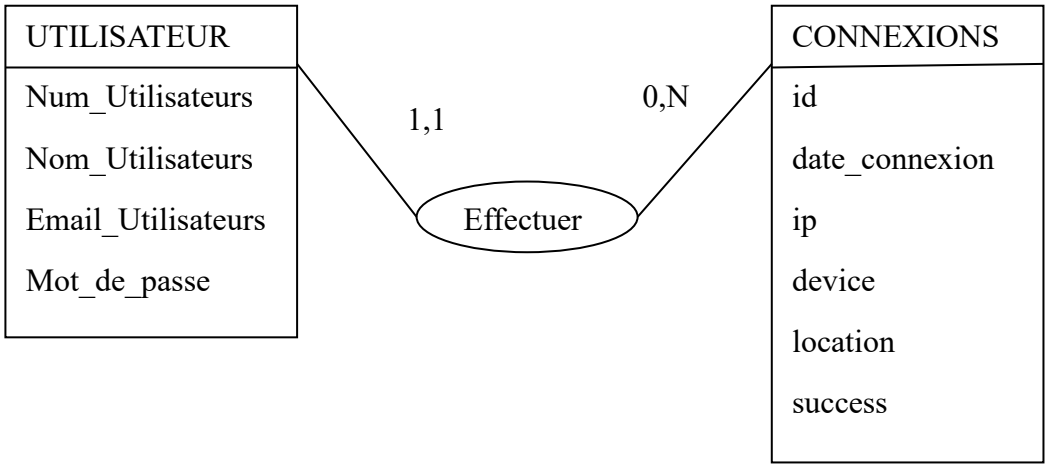

Figure 6.7 : MCD côté sécurité

6.9.3 Les données personnelles

STUDIO
Id_Studio
Adresse
Ville
Code_Postale
Contact
Nif
Stat
Responsable

Figure : 6.8 Tables des données personnelles

6.10 Modèle logique des données (MLD)

Après l'élaboration du modèle conceptuel de données (MCD), la prochaine étape consiste à traduire ce modèle en un **modèle logique de données (MLD)**. Cette transformation permet de définir les structures de données de manière plus précise, en tenant compte des contraintes techniques propres au système de gestion de base de données choisi.

Le MLD sert de passerelle entre la conception conceptuelle et la mise en œuvre physique. Il respecte les règles de normalisation tout en conservant la signification et la cohérence des informations identifiées lors de l'analyse conceptuelle.

Dans ce projet, le MLD a été construit à partir du MCD en appliquant les règles de passage, notamment la transformation des entités et associations en tables, la gestion des cardinalités, et l'intégration des clés primaires et clés étrangères.

6.10.1 Modèle logique de données pour la gestion globale

CLIENTS (<u>Id_Cli</u>, Nom_Cli, Telephon_Cli, Adresse_Cli, Email_Cli).

COMMANDES (Id_Commandes, Date_Commandes, Date_realisation, Date_livraison, #Id_clients).

SERVICES (Num_services, Titre_services, Types_services, Unite_services).

LIGNE_COMMANDE (#Id_commandes, # Num_services, Quantite_services, Prix_unitaire, Total).

6.10.2 Modèle logique de données sur la partie sécurité

UTILISATEURS (Num_Utilisateurs, Nom_Utilisateurs, Email_Utilisateurs, Mot_de_passe).

CONNEXIONS (id, date_connexion, ip, device, location, success, #Num_utilisateurs).

6.10.3 Modèle logique de données sur les informations du studio

STUDIO (Id_Studio, Adresse, Villes, Code_Postale, Contact, Nif, Stat, Responsable).

6.11 Modèle organisationnel de données

Tableau 6.3 Modèle Organisationnel de Données

Tables	Attributs	Responsable	Création	Modification	Suppression	Lecture
CLIENTS	Id_Cli Nom_Cli Telephon_Cli Email_Cli Adresse_Cli	Utilisateur	Oui	Oui	Oui	Oui
COMMANDES	Id_Commandes Date_Commandes Date_realisation Date_livraison Id_Clients	Utilisateur	Oui	Oui	Oui	Oui
SERVICES	Num_services Titre_services Types_services Unite_services	Utilisateur	Oui	Oui	Oui	Oui
LIGNE_ COMMANDE	Quantite_services Prix_unitaire Num_services	Utilisateur	Oui	Oui	Oui	Oui

	Id_Commandes Total					
UTILISATEUR	Num_Utilisateurs Nom_Utilisateurs Email_Utilisateurs Mot_de_passe	Utilisateur	Non	Oui	Non	Non
CONNEXIONS	id Num_Utilisateurs date_connexion ip device location success	Utilisateur	Non	Non	Non	Oui
STUDIO	Id_Studio Adresse Ville Code_Postale Contact Nif Stat Responsable	Utilisateur	Non	Non	Non	Oui

6.12 Modèle physique de données

Après avoir défini la structure conceptuelle, organisationnel et logique des données ainsi que les traitements, la phase de Modèle physique de données constitue l'étape finale avant l'implémentation réelle dans un système de gestion de base données.

6.12.1 Modèle physique de données clients

```
CREATE TABLE clients (

  Id_Cli INT (3) NOT NULL AUTO_INCREMENT,

  Nom_Cli VARCHAR (100) NOT NULL,

  Telephon_Cli VARCHAR (16) NOT NULL,

  Adresse_Cli VARCHAR (100) DEFAULT NULL,
```

```
    Email_Cli VARCHAR (100) DEFAULT NULL,
    PRIMARY KEY (Id_Cli)
);
```

6.12.2 Modèle physique de données services

```
CREATE TABLE services (
    Num_services INT (3) NOT NULL AUTO_INCREMENT,
    Titre_services VARCHAR (100) NOT NULL,
    Types_services VARCHAR (100) NOT NULL,
    Unite_services VARCHAR (100) NOT NULL,
    PRIMARY KEY (Num_services)
);
```

6.12.3 Modèle physique de données commandes

```
CREATE TABLE commandes (
    Id_Commandes INT (3) NOT NULL AUTO_INCREMENT,
    Date_Commandes DATE NOT NULL,
    Date_realisation DATE NOT NULL,
    Date_livraison DATE NOT NULL,
    Id_Clients INT (3) NOT NULL,
    PRIMARY KEY (Id_Commandes),
    FOREIGN KEY (Id_Clients) REFERENCES clients (Id_Cli)
);
```

6.12.4 Modèle physique de données ligne de commandes

```
CREATE TABLE ligne_commande (
    Quantite_services INT (3) NOT NULL,
    Prix_unitaire DECIMAL (15,2) NOT NULL,
```

```
Num_services INT (3) NOT NULL,
Id_Commandes INT (3) NOT NULL,
Total DECIMAL (15,2) AS (Quantite_services * Prix_unitaire) STORED NOT NULL,
PRIMARY KEY (Num_services, Id_Commandes),
CONSTRAINT fk_ligne_commandes_services FOREIGN KEY (Num_services) REFERENCES services (Num_services),
CONSTRAINT fk_ligne_commandes_commandes FOREIGN KEY (Id_Commandes) REFERENCES commandes (Id_Commandes)
) ;
```

6.12.5 Modèle physique de données utilisateur

```
CREATE TABLE utilisateurs (
Num_Utilisateurs INT (3) NOT NULL AUTO_INCREMENT,
Nom_Utilisateurs VARCHAR (100) NOT NULL,
Email_Utilisateurs VARCHAR (100) NOT NULL,
Mot_de_passe VARCHAR (100) NOT NULL,
PRIMARY KEY (Num_Utilisateurs)
);
```

6.12.6 Modèle physique de données connexions

```
CREATE TABLE connexions (
Id INT (11) NOT NULL AUTO_INCREMENT,
Num_Utilisateurs INT (11) DEFAULT NULL,
date_connexion DATETIME DEFAULT CURRENT_TIMESTAMP,
ip VARCHAR (100) DEFAULT NULL,
device VARCHAR (255) DEFAULT NULL,
location VARCHAR (255) DEFAULT NULL,
```

```
    success TINYINT (1) DEFAULT NULL,
    PRIMARY KEY (id),
    KEY idx_num_utilisateurs (Num_Utilisateurs),
    CONSTRAINT fk_connexions_utilisateurs FOREIGN KEY (Num_Utilisateurs)
REFERENCES utilisateurs (Num_Utilisateurs)
);
```

6.12.7 Modèle physique de données studio

```
CREATE TABLE studio (
    Id_Studio INT (3) NOT NULL AUTO_INCREMENT,
    Adresse VARCHAR (100) NOT NULL,
    Ville VARCHAR (100) NOT NULL,
    Code_Postale INT (3) NOT NULL,
    Contact VARCHAR (100) NOT NULL,
    Nif BIGINT (20) NOT NULL,
    Stat BIGINT (20) NOT NULL,
    Responsable VARCHAR (100) NOT NULL,
    PRIMARY KEY (Id_Studio)
) ;
```

La conception du modèle physique de données s’appuie sur une structuration rigoureuse des tables et des relations, garantissant l’intégrité et la cohérence des informations.

Partie III : Réalisation du projet

Chapitre 7 : Spécification des outils de réalisation

7.1 Présentation des outils

7.1.1 Interface utilisateur

L'interface utilisateur (UI) appelé « FRONTEND » correspond à tout ce que l'utilisateur voit et avec quoi il interagit directement dans le navigateur : boutons, formulaires, tableaux, listes, couleurs, styles, polices, etc. C'est la **vitrine** de l'application.

7.1.1 -a React.js

React.js est une bibliothèque JavaScript open source développée par Facebook (aujourd'hui Meta) et sortie en 2013 (des données extrait d'un DOCUMENT EN LIGNE). Elle est utilisée pour créer des UI interactives et dynamiques, notamment dans les applications web modernes. Disponible sur < https://fr.react.dev/ > les documentations sur React.

Contrairement à un Framework complet, React se concentre sur la couche de présentation (le « V » de MVC), laissant aux développeurs le choix autres outils gérer la logique métier ou la communication avec le serveur.

Principes fondamentaux

Composants : React repose sur la création de composants réutilisables. Un composant est une petite partie d'interface (un bouton, un formulaire, un menu …) qui possède son état (state) et ses propriétés (props). Ces composants peuvent être assemblés comme des briques pour construire des interfaces complexes.

Virtual DOM : au lieu de manipuler directement le DOM (Document Object Model) du navigateur, ce qui peut être lent, React utilise un DOM virtuel. Lorsqu'un changement se produit, React compare l'ancienne et la nouvelle version du Virtual DOM, puis met à jour uniquement les parties nécessaires dans le vrai DOM. Résultat de toutes cela, elle permet d'avoir une meilleure performances et réactivité accrue.

Déclaratif plutôt qu'impératif : avec React, on décrit ce que l'on veut voir à l'écran en fonction de l'état des données, et non comment le mettre à jour étape par étape. Cela rend le code plus clair et plus prévisible.

Unidirectional Date Flow (Flux de données unidirectionnel) : les données circulent dans une seule direction du composant parent vers ses enfants. Cette organisation rend l'application plus facile à comprendre et à déboguer.

Utilité

Performance élevée : grâce au Virtual DOM et à une mise à jour optimisée.

Réutilisabilité : un composant créé peut être utilisé dans plusieurs parties du projet.

Large communauté : abondance de ressources, bibliothèques, et documentation.

Ecosystème riche : intégration facile avec d'autre outils comme Redux, Next.js ou React Router.

Maintenance facilitée : grâce à une architecture claire.

7.1.1 -b Tailwind CSS

Tailwind CSS est un Framework CSS utilitaire (utility-first) open-source créé par Adam Wathan et lancé en 2017. Il se distingue des Framework CSS traditionnels (comme Bootstrap ou Foundation) en offrant une approche où l'on construit les interfaces directement à l'aide de classes utilitaires prédéfinies, plutôt que de s'appuyer sur des composants préfabriqués.

L'objectif de Tailwind CSS est de permettre aux développeurs de concevoir des interfaces entièrement personnalisées tout en bénéficiant de la rapidité et de la cohérence qu'offre un système de classe réutilisables.

Principes fondamentaux

Approche Utility-First : au lieu d'écrire in fichier CSS séparé avec des règles personnalisées, on utilise des classes courtes directement dans le HTML ou JSX.

Personnalisation totale : Tailwind ne force pas aucun style visuel par défaut ; il fournit juste une base de styles utilitaires. On peut configurer la palette de couleurs, les espacements, les polices, etc. via un fichier « tailwind.config.js ».

Responsive Design intégré : Tailwind facilite la création de sites responsives grâce à des préfixes mobiles et desktop.

Performances grâce au « Purge » : Tailwind supprime automatiquement les classes inutilisées lors de ma compilation en production, réduisant la taille finale du CSS.

Utilité

Rapidité de développement : pas besoin d'écrire du CSS à partir de zéro.

Flexibilité totale : contrairement à Bootstrap, aucun style imposé.

Cohérence visuelle : les classes utilitaires garantissent une uniformité dans le design.

Facilité de maintenance : tout le style est directement visible dans HTML/JSX, ce qui évite de chercher dans plusieurs fichiers.

Excellent pour les projets modernes : particulièrement adapté aux environnement React, Vue.js, Angular.

7.1.1 -c TypeScript

TypeScript est un langage de programmation open-source développé par Microsoft et lancé en 2012. Il est à la fois JavaScript avec de nouvelles fonctionnalités, tout en restant compatible avec celui-ci. La principale caractéristique de TypeScript est l'ajout du typage statique, qui permet de détecter des erreurs dès la phase de compilation, avant même l'exécution du code.

<u>Principes fondamentaux</u>

Typage statique : dans JavaScript, les types (nombre, chaîne, tableau, etc.) sont vérifiés au moment de l'exécution. En TypeScript, les types sont vérifiés avant l'exécution, ce qui permet de détecter les erreurs plus tôt.

Compatibilité avec JavaScript : tout code JavaScript valide est également valide en TypeScript. On peut donc migrer un projet JS vers TS progressivement.

Fonctionnalité avancée : interface, classes et héritages, types personnalisés et génériques, modules et import/export.

Compilation JavaScript : le navigateur ne comprend pas directement TypeScript. Le TS est transpilé en JavaScript grâce au compilateur TSC.

<u>Utilité</u>

Moins d'erreurs : les bugs liés aux types sont détectés avant l'exécution.

Lisibilité accrue : le typage rend le code plus clair pour l'équipe.

Maintenance facilitée : utile pour les grands projets ou le travail d'équipe.

Interopérabilité : fonctionne avec toutes les bibliothèques JavaScript existantes.

TypeScript s'impose comme un outil essentiel dans le développement JavaScript moderne, en particulier pour les projets de grande envergure. Grâce à son typage statique, il améliore la qualité du code, réduit les erreurs et facilite la collaboration. En étant entièrement compatible avec JavaScript, il permet une transition progressive et sans risque, tout en offrant des fonctionnalités avancées qui renforcent la robustesse et la maintenabilité des applications.

7.1.1 -d Axios

Axios est une bibliothèques JavaScript open-source permettant d'effectuer des requêtes HTTP depuis un navigateur ou un environnement Node.js. Elles sont principalement utilisées pour communiquer avec des API (Application Programming Interface) et récupérer ou envoyer des données à un serveur.

Axios est souvent choisi pour sa simplicité, sa syntaxe claire et ses fonctionnalités avancées par rapport à l'API native fetch () de JavaScript.

<u>Principes fondamentaux</u>

Requêtes HTTP simplifiées : Axios permet d'envoyer facilement des requêtes GET, POST, PUT, DELETE.

Support du JSON par défaut : les données envoyées et reçues sont automatiquement converties en JSON, ce qui évite les conversions manuelles.

Comparabilité navigateur et Node.js : Axios fonctionne à la fois côté client (navigateur) et côté serveur (Node.js).

Fonctionnalités avancées : intercepteurs pour modifier les requêtes ou les réponses avant qu'elles ne soient traitées ; annulation de requêtes ; gestion automatique des en-têtes HTTP ; support des requêtes asynchrones avec asyn/await.

<u>Utilité</u>

Syntaxe simple et lisible comparée à fetch ().

Support natif des promesses et de async/await.

Gestion centralisée des erreurs.

Plus de fonctionnalités (intercepteurs, configuration globale, annulation de requêtes, etc.).

Compatible avec les anciens navigateurs.

Axios est devenu l'un des outils les plus populaires pour gérer les requêtes HTTP dans les projets JavaScript modernes. Grâce à sa simplicité, sa compatibilité large et ses nombreuses fonctionnalités, il facilite grandement l'intégration avec des API et la gestion des échanges de données. Que ce soit dans un projet React, Vue.js, Node.js ou même pur JavaScript, Axios s'intègre parfaitement et améliore la productivité du développeur.

7.1.1 -e Vite

Vite est un outil de développement web moderne conçu pour créer des applications frontend rapides et performantes. Il se distingue par un démarrage quasi instantané grâce au module ES natif et un rechargement à chaud (HMR) ultra-rapide pendant le développement. Vite prend en charge les Framework modernes comme React, Vue ou Svelte, et compile le code pour la production de manière optimisée avec Rollup. Son objectif est de simplifier le workflow des développeurs tout en offrant des performances maximales, surtout pour les projets web modernes.

7.1.2 Backend

Le Backend désigne la partie invisible d'une application ou d'un site web, c'est-à-dire l'ensemble des processus et logiques qui s'exécutent côté serveur. Il est responsable du traitement des données, de la gestion des requêtes des utilisateurs, de la communication avec la base de données, ainsi que de la sécurité et de l'authentification.

7.1.2 -a Node.js

Node.js est un environnement d'exécution JavaScript côté serveur, open-source, créé en 2009 par Ryan Dahl.

Il permet d'exécuter du code JavaScript en dehors du navigateur, grâce au moteur V8 de Google Chrome.

Avec Node.js, JavaScript n'est plus limité au développement frontend : il peut désormais être utilisé pour construire des applications backend rapides, scalables et performantes.

<u>Caractéristique principale</u>

Asynchrone et orienté événements : Node.js utilise un modèle non bloquant qui permet de gérer plusieurs requêtes simultanément sans attendre de la fin d'une tâche.

Basé sur le moteur V8 : le moteur V8 compile le JavaScript en code machine, offrant ainsi des performances élevées.

Monothread mais hautement concurrent : Node.js fonctionne sur un seul thread principal mais peut gérer un grand nombre de connexions grâce à sa boucle d'éléments.

Écosystème riche : grâce à **npm** (Node Package Manager), Node.js dispose de milliers de modules et bibliothèques prêts à l'emploi.

Utilité

Performance et rapidité grâce à son architecture asynchrone.

Utilisation d'un seul langage (JavaScript) pour le Frontend et le Backend.

Scalabilité : adapté aux applications en temps réel.

Large communauté et documentation abondante.

En somme, Node.js a révolutionné le développement web en permettant d'utiliser JavaScript côté serveur. Sa rapidité, son modèle asynchrone et son écosystème riche en font un choix privilégié pour la création d'API, d'applications en temps réel et de services web modernes.

7.1.2 -b Express.js

Express.js (ou simplement Express) est un Framework web minimaliste pour Node.jsn créé en 2010 par TJ Holowaychuk**.** Il simplifie le développement d'applications et d'API en fournissant une structure légère et flexible pour gérer les requêtes HTTP, les routes, et les middlewares. Express est aujourd'hui l'un des Framework les plus populaires dans l'écosystème Node.js, souvent utilisé pour créer des applications Backend rapides et modulaires.

Caractéristiques principales

Minimaliste mais extensible : Expresse n'impose de structure rigide ; il fournit juste les outils essentiels et laisse au développeur la liberté de structurer son projet.

Gestion des routes : Express permet de définir facilement les routes pour répondre à différentes URL et méthodes HTTP.

Middlewares puissants : les middlewares sont des fonctions qui traitent les requêtes avant qu'elles n'arrivent à la réponse finale.

Support JSON et formulaires : Expresse gère facilement l'envoi et la réception de données format JSON ou application/x-www-form-urlencoded.

Utilité

Simplicité et rapidité de développement.

Flexibilité pour créer des architectures adaptées aux besoins.

Large écosystème : compatibles avec des milliers de modules npm.

Intégration facile avec des base de données.

Idéal pour les API REST et les applications web.

Express.js est un outil incontournable pour les développeurs Node.js qui souhaitent créer rapidement des applications web ou des API robustes et modulaires. Sa simplicité, combinée à sa flexibilité et à son vaste écosystème, en fait l'un des Framework les plus utilisés dans le développement Backend moderne.

7.1.3 Autre outil

7.1.3 -a Concurrently

Concurrently est un outil en ligne de commande (CLI) open-source qui permet d'exécuter plusieurs scripts ou commandes en parallèle dans un même terminal. Il est particulièrement utilisé dans les projets JavaScript / Node.js ; par exemple lorsque l'on souhaite lancer simultanément un serveur backend et un client frontend pendant le développement.

Caractéristique principale

Exécution simultanée : permet de lancer plusieurs scripts définis dans un fichier package.json sans avoir à ouvrir plusieurs terminaux.

Affichage clair : chaque commande exécutée affiche ses logs dans le terminal avec un préfixe coloré pour les distinguer.

Compatible multi-plateformes : fonctionne sur Windows, MacOs, et Linux sans configuration supplémentaire.

Flexible et personnalisable : permet de configurer l'ordre d'exécution, les préfixes, les couleurs, et même d'arrêter toutes les commandes si l'une échoue.

Utilité

Gain de temps : plus besoin de lancer plusieurs terminaux pour chaque service.

Pratique pour le développement full-stack : par exemple, démarrer un serveur Node.js (Backend) et une application React (Frontend) en une seule commande.

Lisibilité : les journaux sont clairement séparés par couleurs et préfixes.

Automatisation : simplifie les scripts npm pour le développement.

7.1.3 -b GitHub

GitHub est une plateforme en ligne de gestion et d'hébergement de code source qui utilise le système de contrôle de version Git.

Git est l'outil qui permet de suivre les modifications dans un projet (ajouts, suppressions, améliorations de code). Disponible sur un tutoriel pour l'apprentissage de GitHub < https://youtu.be/FDjJA4OJcto >.

GitHub, lui, ajoute une dimension collaborative en permettant à plusieurs développeurs de travailler ensemble, de partager du code et de coordonner leurs contributions.

Utilité

Hébergement de code : stocker et sauvegarder le code d'un projet dans le cloud.

Travail collaboratif : plusieurs développeurs peuvent travailler sur le même projet sans écraser le travail des autres.

Versioning : garder l'historique des changements, revenir à une version antérieure en cas de bug.

Pull Requests et Revue des codes : proposer des modifications, discuter entre développeurs avant intégration.

Open source : des millions de projets sont publics, permettant à d'autres développeurs d'apprendre, contribuer ou réutiliser du code.

CI/CD (Intégration Continue/ Déploiement Continu) : GitHub peut être relié à des outils pour tester et déployer automatiquement une application.

Avantages

Centralise le travail d'une équipe.

Facilite la gestion des versions et des branches (par exemple : branche main, branche develop, branche master, branche feature).

Permet de collaborer partout dans le monde.

Sert aussi de portfolio professionnel pour montrer ses projets (souvent demandé en entretien technique).

Le projet a été hébergé sur GitHub toute au long de son développement (Voir annexe A page 76)

7.1.4 Base de données

MariaDB est un système de gestion de base de données relationnelles (SGBDR) open-source, issu d'un fork de MySQL. Il a été créé en 2009 par Michael Monty Widenius, l'un des fondateurs de MySQL, à la suite du rachat de ce dernier par Oracle Corporation. L'objectif de MariaDB est de garantir la continuité, la performance et la liberté d'utilisation du code source, tout en restant totalement compatible avec MySQL.

Caractéristiques principales

Compatibles avec MySQL : les commandes, les API et les formats de données MariaDB sont identiques à ceux de MySQL, ce qui facilite la migration.

Performance et optimisation : MariaDB introduit de nouveaux moteurs de stockage et optimise certaines requêtes pour un traitement plus rapide.

Open-source : Développé par la communauté et sous licence GPL, garantissant une transparence totale.

Sécurité renforcée : améliorations de la gestion des droits utilisateurs, support de l'authentification par plugin et chiffrement avancé.

Utilité

Gratuit et open-source sans restriction propriétaires.

Performant et évolutif, adapté aux petites comme grandes applications.

Compatibles avec MySQL, ce qui facilite la migration.

Large communauté et documentation complète.

Multi-plateforme.

7.1.5 Outils logiciels

7.1.5 -a Visual Studio Code

Visual Studio Code, souvent abrégé en VSC ou VS Code, est un éditeur de code open source développé par Microsoft. Il est conçu pour être léger, rapide et extensible. VS Code prend en charge un grand nombre de langages de programmation grâce à ses extensions et offre des fonctionnalités avancées telles que :

- La coloration syntaxique pour rendre le code plus lisible.
- L'autocomplétions intelligente pour suggérer des mots-clés, fonctions et variables.
- Le débogage intégré pour détecter et corriger les erreurs directement dans l'éditeur.
- L'intégration Git permettant de gérer la version du code sans quitter l'éditeur.

Son interface moderne et sa communauté très active en font l'un des outils préférés des développeurs dans de nombreux domaines, du développement web au développement logiciel.

7.1.5 -b Xampp

Xampp est un environnement de développement gratuit open source qui regroupe plusieurs outils essentiels pour héberger localement un site web ou une application (voir annexe B page 76). Il inclut :

- Apache : serveur web permettant de diffuses des pages web aux navigateurs.
- MariaDB : système de gestion de bases de données relationnelles.
- PHP et Perl : langages de programmation côté serveur.

L'avantage de Xampp est qu'il permet de créer de serveur local sur son ordinateur, idéal pour tester un site ou une application web sans avoir besoin d'un hébergement en ligne. Son interface simple permet de démarrer ou arrêter facilement les services nécessaires.

7.1.5 -c Google Chrome

Google Chrome est un navigateur web développé par Google, réputé pour sa rapidité, sa sécurité et sa simplicité d'utilisation. Il est particulièrement apprécié des développeurs pour :

- Ses outils de développements intégrés (DevTools) permettant d'inspecter le code HTMLL, CSS et JavaScript d'une page.
- La possibilité de tester des sites en mode responsive pour simuler différents écrans (ordinateur, téléphone, tablette).
- Ses extensions qui facilitent le développement, comme les inspecteurs de couleur, les simulateurs d'API ou les bloqueurs de cache.

Chrome est donc non seulement un outil de navigation, mais aussi un indispensable pour les développeurs et le débogage web.

7.2 Justification des choix

7.2.1 Frontend : TypeScript, React, Tailwind CSS, Vite

Pour le développement de la partie Frontend, plusieurs technologies modernes ont été sélectionnées afin de garantir la performance, la maintenabilité et la simplicité d'évolution du projet.

React a été choisi pour sa popularité, sa structure basée sur les composants réutilisables et sa grande communauté.

TypeScript apporte la robustesse du typage statique, réduisant les erreurs et facilitant la collaboration sur des projets complexes.

Tailwind CSS permet une conception d'interface rapide et moderne, grâce à une approche utilitaire qui évite la surcharge de fichiers CSS traditionnels.

Enfin, Vite offre un environnement de développement rapide avec un démarrage instantané et un rechargement à chaud performant, ce qui améliore l'efficacité des développeurs.

Ces choix technologiques permettent de construire une application fiable, moderne et évolutive, tout en facilitant la productivité des développeurs.

Tableau comparatif des technologies frontend

Tableau 7.1 Comparatif côté frontend

Critère	React	Vue.js / Angular	Avantage React
Popularité et communauté	Très élevée, écosystème riche	Vue populaire mais plus jeune / Angular	Large soutien et documentation

		plus complexe	
Facilité d'apprentissage	Relativement simple	Vue facile / Angular plus lourd	Courbe d'apprentissage équilibré
Réutilisabilité	Composants modulaires	Idem mais Angular impose plus de règles	Flexibilité sans rigidité
Critère	**TypeScript**	**JavaScript classique**	**Avantage de TypeScript**
Robustesse	Typage statique, moins d'erreurs	Dynamique, plus d'erreurs possibles	Meilleure maintenabilité
Lisibilité de code	Plus claire et structurée	Moins strict, le code parfois confus	Code plus sûr et lisible
Adaptation à long terme	Favorisé dans les grands projets	Bien mais moins rouste	Idéal pour l'évolutivité
Critère	**Tailwind CSS**	**Bootstrap / CSS classique**	**Avantage Tailwind**
Rapidité de développement	Très rapide avec classes utilitaires	Bootstrap rapide mais plus rigide / CSS pur plus lent	Flexibilité et personnalisation
Modernité	Très moderne, tendance actuelle	Bootstrap connu mais moins flexible	Design moderne et réactif
Poids et optimisation	CSS généré optimisé	Bootstrap plus lourd	Les performances s'accumulent
Critère	**Vite**	**Web pack / Créer une application React**	**Avantage de Vite**
Temps de démarrage	Instantanée grâce à ES Modules	Web pack plus lent, CRA lourd	Gain de temps pour le développement
Rechargement à chaud (HMR)	Ultra rapide	Web pack plus carême	Expérience de développement fluide
Simplicité	Configuration minimaliste	Complexe de configuration Web pack	Mise en place facile

Ce tableau montre clairement que les choix React + TypeScript + Tailwind + Vite offrent un équilibre optimal entre productivité, performance et maintenabilité, ce qui justifie leur adoption pour ce projet. Disponible sur < https://youtu.be/-jnV9cNSCS8 >.

7.2.2 Backend: Node.js, Express.js, MariaDB

Pour la partie Backend, plusieurs technologies modernes ont été retenues afin d'assurer une rapidité de développement, une sécurité accrue, et une bonne performance dans la gestion des données et des communications entre le client et le serveur.

Node.js a été choisi comme environnement d'exécution car il permet d'exécuter du JavaScript côté serveur avec un excellente performance et une gestion asynchrone des requêtes.

Express.js, Framework minimaliste basé sur Node.js, facilite la création d'API et de routes, tout en restant flexible et léger.

TypeScript a été utilisé pour renforcer la fiabilité du code grâce au typage statique, rendant le développement plus robuste et moins sujet aux erreurs. Disponible sur le tutoriel < https://youtu.be/a3nj66pzKJ0 >.

MariaDB a été retenue comme système de gestion de base de données relationnelle pour sa compatibilité avec MySQL, sa stabilité et ses performances élevées dans la gestion de grandes quantités de données.

Axios est intégré afin de faciliter la communication entre le frontend et le backend grâce à des requêtes HTTP simples et efficaces.

Enfin Concurenttly a été adopté pour exécuter simultanément le serveur et le client lors su développement, ce qui simplifie considérablement le workflow des développeurs.

Ces choix garantissent un écosystème complet, cohérent et moderne, assurant la performance, la maintenabilité et la facilité d'intégration entre les différentes couches de l'application.

Tableau comparatif et justification des choix

Tableau 7.2 Comparatif côté backend

Technologie	Rôle	Avantages	Alternatives possibles	Justification des choix

Node.js	Exécution JavaScript côté serveur	Rapide (moteur V8), asynchrone, large communauté	Python (Django / Flask), PHP (Laravel), Java (Spring Boot)	Idéal pour unifier le langage front/back et gérer des API performantes
Express.js	Framework basé sur Node.js pour créer des API REST	Léger, flexible, middleware puissant, grande communauté	Kao.js, NestJS, Django REST, Laravel	Plus simple et rapide à mettre en place, parfait pour les API REST
TypeScript	Surcouche typée de JavaScript	Sécurité des types, meilleure lisibilité, maintenabilité	JavaScript pur, Flow	Réduit les erreurs et facilite le travail en équipe
MariaDB	Base de données relationnelle	Open-source, compatible MySQL, stable et performante	MySQL, PostgreSQL, MongoDB	Solide, gratuit et adapté aux projets nécessitant des relations
Axios	Client HTTP pour échanges API	Simple, support async/await, gestion des erreurs	Fetch API, SuperAgent	Plus pratique et lisible que fetch, standard de facto en React
Concurrently	Gestion des scripts multiples	Lance plusieurs processus en parallèle, simple d'usage	PM2, Nodemon + scripts séparés	Pratique pour lancer front et back en même temps en développement

En somme, le choix des technologies utilisées dans ce projet repose sur un équilibre entre modernité, performances et simplicité d'utilisation. Côté frontend, l'adaptation de React combiné à TypeScript, Tailwind CSS et Vite a permis de mettre en place une interface à la fois intuitive, rapide et maintenable, tout en respectant les bonnes pratiques du développement

moderne. Côté backend, l'association de Node.js, Express, TypeScript et MariaDB garantit un environnement fiable, flexible et performant, adapté aux besoins d'une architecture orientée services.

L'utilisation d'Axios facilite les échanges entre client et le serveur, tandis que Concurrently simplifie le lancement et la synchronisation des différents services, optimisant ainsi le cycle de développement.

Ainsi, l'ensemble des choix technologiques permet de construire une application robuste, évolutive et performante, répondant aux exigences d'un projet académique tout en étant alignée sur les standards industriels actuels. Ces choix offrent également une base solide pour de futures évolutions, aussi bien en termes de fonctionnalités que de montée en charge.

7.2.3 Logiciels : Visual Studio Code, Xampp, Google Chrome

L'utilisation des logiciels Visual Studio Code, Google Chrome et Xampp repose sur leur simplicité, performance et compatibilité avec les technologies modernes. Ces outils constituent un environnement de travail complet allant de la conception au test et à l'exécution de l'application.

VSC a été choisi pour le développement efficace grâce à sa légèreté et son écosystème d'extensions.

Chrome pour le test et le débogage grâce à ses outils intégrés performants pour le développement.

Xampp pour la gestion simple de base de données et la mise en place rapide d'un serveur local.

Tableau comparatif des logiciels

Tableau 7.3 Comparatif des logiciels

Logiciel	Alternatives possibles	Avantages des choix
Visual Studio Code	Sublime Text, Atom, IntelliJ IDEA	Léger, open source, très populaire, extensions riches, intégration Git
Google Chrome	Mozilla Firefox, Microsoft Edge, Safari	Outils de développement performants, compatibilité avec la majorité des

		Framework modernes, rapidité d'exécution
Xampp	WAMP, MAMP, Docker	Installation simple, configuration rapide, support de MariaDB, idéal pour un projet académique.

Brièvement, le choix des logiciels s'est orienté vers des outils pratiques, largement adoptés et adaptés au contexte du projet. Visual Studio Code a été retenu comme environnement de développement intégré pour sa légèreté, ses nombreuses extensions et son support complet de TypeScript, React et Node.js. Google Chrome a été utilisé comme navigateur principal afin de tester, déboguer et analyser l'application web, grâce à ses outils de développement puissants et intuitifs. Enfin, Xampp a servi pour la gestion de la base de données MariaDB, offrant un environnement simple et rapide à configurer, idéal pour un projet académique.

Chapitre 8 : Mise en œuvre et implémentation

La mise en œuvre de notre projet a été organisée en plusieurs étapes afin d'assurer une structure claire et une meilleure gestion du développement. Dans un premier temps, nous avons travaillé sur la partie Frontend, qui constitue l'interface utilisateur et permet l'interaction directe avec le système. Ensuite, nous avons développé la partie Backend, qui gère la logique métier et la communication avec la base de données. Enfin, pour faciliter l'exécution simultanée des deux environnements, nous avons utilisé Concurrently, un outil qui permet de lancer le front et le back en parallèle, garantissant une meilleure fluidité lors des tests et de l'intégration.

8.1 L'implémentation côté frontend

Après l'installation des différentes outils utilisés dans la conception et réalisation du projet, nous avons entamé le premier codage du code ; c'est le code de l'authentification dont le figure ci-dessous représente l'extrait du code.

```
<h2 className={`text-xl font-bold ${darkMode ? 'text-white' : 'text-pink-600'}`}>
  Authentification
</h2>

{/* Email */}
<div className="space-y-1">
  <input
    type="email"
    placeholder="Adresse email"
    {...register('email')}
    onInput={(e) => {
      const input = e.currentTarget;
      input.value = input.value.replace(/[^\w@.\-_]/gi, '');
    }}
    className={`w-full px-5 py-3 rounded-full border shadow-inner text-sm bg-transparent outline-none ${
      darkMode
        ? 'border-purple-500 text-white placeholder-gray-400'
        : 'border-pink-300 text-black placeholder-gray-500'
    }`}
  />
  {errors.email && <p className="text-red-500 text-xs">{errors.email.message}</p>}
</div>

{/* Mot de passe */}
<div className="space-y-1 relative">
  <input
    type={showPassword ? 'text' : 'password'}
    placeholder="Mot de passe"
    {...register('password')}
```

Figure 8.1 Extrait de code de l'authentification côté frontend

Ensuite, la programmation a continué avec la création du contenue de la navbar, pour faciliter le UX (Expérience utilisateur).

```
    }`}
  >
    <img src="/logo.png" alt="Logo" className="w-11 h-9" />
    Merix Studio
  </Link>

  {/* Menu Desktop */}
  <ul className="hidden min-[1200px]:flex items-center gap-10 font-medium text-sm">
    <li><Link to="/accueil" className={linkStyle}><Home size={18} /> Accueil</Link></li>
    <li><Link to="/clients" className={linkStyle}><Users size={18} /> Clients</Link></li>
    <li><Link to="/services" className={linkStyle}><Music size={18} /> Services</Link></li>
    <li><Link to="/commandes" className={linkStyle}><Package size={18} /> Commandes</Link></li>
    <li><Link to="/lignes-commandes" className={linkStyle}><FileText size={18} /> Lignes de commande</Link></li>
    <li><Link to="/parametres" className={linkStyle}><Settings size={18} /> Paramètres</Link></li>
    <li>
      <button onClick={handleLogout} className={linkStyle}>
        <LogOut size={18} /> Déconnexion
      </button>
    </li>
  </ul>

  {/* Mobile Menu Toggle */}
  <button
    onClick={toggleMenu}
    className={`min-[1200px]:hidden focus:outline-none transition-transform duration-200 ${
      darkMode ? "text-purple-300 hover:text-purple-500" : "hover:text-[#f472b6]"
```

Figure 8.2 Extrait du code de navigation

En outre, le développement a évalué dans la création des pages pour la fonctionnalité principale de l'application.

L'accueil contient les recettes annuelles et mensuelles de l'application.

```
  <h2 className={`text-2xl font-bold tracking-wide ${
        darkMode ? "text-purple-300" : "text-[#ec4899]"
      }`}>
    Recette annuelle totale
  </h2>
  <p className="text-2xl font-bold drop-shadow-lg whitespace-pre-wrap">
    {recetteAnnuelle.toLocaleString()} Ar
  </p>
</div>

{/* Résultats */}
{recettesTriees.length === 0 ? (
  <p
    className={`text-center text-lg mt-12 italic font-semibold transition-colors duration-500 ${
      darkMode ? "text-purple-300" : "text-pink-400"
    }`}
  >
    Aucune recette trouvée avec les filtres actuels.
  </p>
) : (
  <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-8">
    {recettesTriees.map((r, i) => (
      <div
        key={i}
        className={`p-6 rounded-2xl border shadow-lg transition-colors duration-500 hover:shadow-2xl ${
```

Figure 8.3 Extrait du code de l'accueil

Puis les CRUD des clients, services, commandes et lignes de commandes avec des fonctionnalités supplémentaires : filtrage dynamique pour effectuer une recherche en temps réelles des données dans chaque liste. Voici un extrait de code client pour représenter les CRUD dans l'application.

```
const Clients = () => {
  const ajouterOuModifierClient = async (e: React.FormEvent) => {
      setClient({ nom: "", telephone: "", adresse: "", email: "" });
      setMessageErreur("");
    } catch (err) {
      console.error("Erreur d'enregistrement :", err);
      setMessageErreur("Erreur lors de l'enregistrement.");
    }
  };

  const modifierClient = (index: number) => {
    const clientAModifier = clients[index];
    setClient({ ...clientAModifier });
    setIndexModification(index);
     setErreurTelephone("");
    setErreurEmail("");
    setMessageErreur("");
  };

  const annulerModification = () => {
    setClient({ nom: "", telephone: "", adresse: "", email: "" });
    setIndexModification(null);
    setErreurTelephone("");
    setErreurEmail("");
    setMessageErreur("");
  };

  const supprimerClient = async (index: number) => {
    if (!window.confirm("Confirmer la suppression de ce client ?")) return;
    const id = clients[index].id;
    try {
      await axios.delete(`http://localhost:5000/api/clients/${id}`);
```

Figure 8.4 Extrait de code représentant un CRUD

Cependant, dans la partie lignes de commandes, il y a aussi la fonctionnalité qui permet de faire une exportation PDF des factures.

```
const factureRef = useRef<HTMLDivElement>(null);

// Impression via react-to-print
const handlePrint = useReactToPrint({
  contentRef: factureRef,
});

const normalize = (str: string) =>
str
  .normalize("NFD")
  .replace(/\p{Diacritic}/gu, "")
  .replace(/\s+/g, " ")
  .trim()
  .toLowerCase();

// Imprime dès que factureActive est défini
useEffect(() => {
  if (factureActive) {
    handlePrint();
  }
}, [factureActive, handlePrint]);

useEffect(() => {
  chargerCommandes();
  chargerServices();
  chargerLigneCommandes();
  chargerStudio();
}, []);
```

Figure 8.5 Extrait du code de l'exportation PDF

Enfin, pour moderniser l'application, des fonctionnalités supplémentaires ont été ajouté. Premièrement la gestion de thème permettant de basculer en mode sombre et clair.

```
const DarkModeContext = createContext<DarkModeContextProps | undefined>(undefined);

export const DarkModeProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
  const [darkMode, setDarkMode] = useState<boolean>(() => {
    const saved = localStorage.getItem("darkMode");
    return saved ? JSON.parse(saved) : false;
  });

  useEffect(() => {
    localStorage.setItem("darkMode", JSON.stringify(darkMode));
  }, [darkMode]);

  const toggleDarkMode = () => setDarkMode((prev) => !prev);

  return (
    <DarkModeContext.Provider value={{ darkMode, toggleDarkMode }}>
      {children}
    </DarkModeContext.Provider>
  );
};

export const useDarkMode = (): DarkModeContextProps => {
  const context = useContext(DarkModeContext);
  if (!context) {
    throw new Error("useDarkMode must be used within a DarkModeProvider");
  }
  return context;
};
```

Figure 8.6 Extrait de code pour le thème

En plus, le paramètre contenant les informations personnelles du studio, une fonctionnalité permettant à l'utilisateur de changer son mot de passe ; puis la côté sécurité affichant les listes de connexions effectuées dans l'application, suivi des historiques de commandes et les notifications.

```
const tabs = [
  { id: "profil", label: "Informations personnelles", icon: <Info /> },
  { id: "motdepasse", label: "Mot de passe", icon: <Lock /> },
  { id: "securite", label: "Sécurité ", icon: <Shield /> },
  { id: "preferences", label: "Préférences", icon: <Settings /> },
  { id: "notifications", label: "Notifications", icon: <Bell /> },
  { id: "historique", label: "Historique ", icon: <Clock /> },
];
const { darkMode, toggleDarkMode } = useDarkMode();

// Chargement des commandes si l'onglet actif est "historique"
useEffect(() => {
  if (activeTab === "historique") {
    axios
      .get("http://localhost:5000/api/commandes")
      .then((res) => {
        setCommandes(res.data);
      })
      .catch((err) => {
        console.error("Erreur chargement commandes :", err);
        setCommandes([]);
      });
  }
}, [activeTab]);

const [showNotifications, setShowNotifications] = useState(false);

// Type des notifications
type Notification = {
```

Figure 8.7 Extrait de code des fonctionnalités dans le paramètre

8.2 L'implémentation côté backend

Dans la logique de développement, le backend de l'authentification est la première code mis en œuvre.

```
export const login = async (req: Request, res: Response): Promise<void> => {
  const { email, password } = req.body;

  try {
    const [rows] = await db.execute(
      'SELECT * FROM Utilisateurs WHERE Email_Utilisateurs = ?',
      [email]
    );

    const user = Array.isArray(rows) && rows.length > 0 ? (rows[0] as User) : null;

    const rawIp = req.headers['x-forwarded-for'] || req.socket.remoteAddress || '';
    const ip = Array.isArray(rawIp) ? rawIp[0] : rawIp.toString().split(',')[0].trim();
    const agent = useragent.parse(req.headers['user-agent'] || '');
    const device = `${agent.os} - ${agent.toAgent()}`;

    let location: string;
    if (/^(::1|127\.0\.0\.1|::ffff:127\.0\.0\.1)$/.test(ip)) {
      location = 'Localhost, Dev';
    } else {
      const geo = geoip.lookup(ip);
      location = geo && (geo.city || geo.country)
        ? `${geo.city || 'Ville inconnue'}, ${geo.country || 'Pays inconnu'}`
        : 'Inconnue';
    }

    if (!user) {
      // Tentative échouée : email non trouvé
      await db.execute(
        `INSERT INTO connexions (Num_Utilisateurs, ip, device, location, success)
         VALUES (?, ?, ?, ?, ?)`,
        [null, ip.toString(), device, location, false]
```

Figure 8.8 Extrait de code backend de la connexion

Ensuite, les codes se sont enchaînés suivant les processus du côté frontend. Dont Voici un extrait de code pour représenter les CRUD côté backend.

```
export const createService = async (req: Request, res: Response) => {
  } catch (err) {
    console.error("Erreur createService:", err);
    res.status(500).json({ message: "Erreur serveur" });
  }
};

export const updateService = async (req: Request, res: Response) => {
  const id = req.params.id;
  const { titre, type, unite } = req.body as Service;
  try {
    await db.execute(
      "UPDATE services SET Titre_services = ?, Types_services = ?, Unite_services = ? WHERE Num_services = ?",
      [titre, type, unite, id]
    );
    res.status(200).json({ message: "Service mis à jour" });
  } catch (err) {
    console.error("Erreur updateService:", err);
    res.status(500).json({ message: "Erreur serveur" });
  }
};

export const deleteService = async (req: Request, res: Response) => {
  const id = req.params.id;
  try {
    await db.query('DELETE FROM ligne_commande WHERE Num_services = ?', [id]);
    await db.execute("DELETE FROM services WHERE Num_services = ?", [id]);
    res.status(200).json({ message: "Service supprimé" });
  } catch (err) {
    console.error("Erreur deleteService:", err);
    res.status(500).json({ message: "Erreur serveur" });
  }
```

Figure 8.9 Extrait des CRUD côté backend

L'implémentation du Backend a permis de poser les fondations logiques et fonctionnelles de notre application. À travers la mise en place des routes, de la gestion des données et des échanges sécurisés avec la base, nous avons assuré la cohérence et la fiabilité du système. Ce travail constitue le cœur invisible de l'application, garantissant le bon fonctionnement de l'ensemble et permettant une interaction fluide avec le Frontend. Ainsi, le Backend joue pleinement son rôle de pilier technique, sur lequel reposent les performances et la robustesse de notre solution

Chapitre 9 : Présentation de l'application développée

L'application développée dans le cadre de ce stage vise à répondre aux besoins identifiés lors de la phase d'analyse. Elle constitue une solution pratique et adaptée, combinant simplicité d'utilisation pour l'utilisateur final et efficacité technique. À travers une interface claire et une architecture bien structurée, elle permet d'illustrer concrètement les choix technologiques retenus ainsi que leur intégration cohérente. La présentation qui suit met en évidence les principales fonctionnalités de l'application, son organisation et son apport par rapport aux objectifs fixés.

9.1 Présentation de la page d'authentification

Les informations que l'utilisateur a besoin sont son adresse email et mot de passe. Il possède 5 tentatives pour éviter le blocage pendant 30 secondes. Ensuite il lui restera des tentatives limitées pour que l'application soit définitivement bloquée.

L'utilisateur a aussi l'avantage d'utiliser la fonction « se souvenir moi » pour afficher automatiquement l'adresse email aux prochaines connexions. Enfin, il peut afficher et masquer le mot de passe lors de l'authentification.

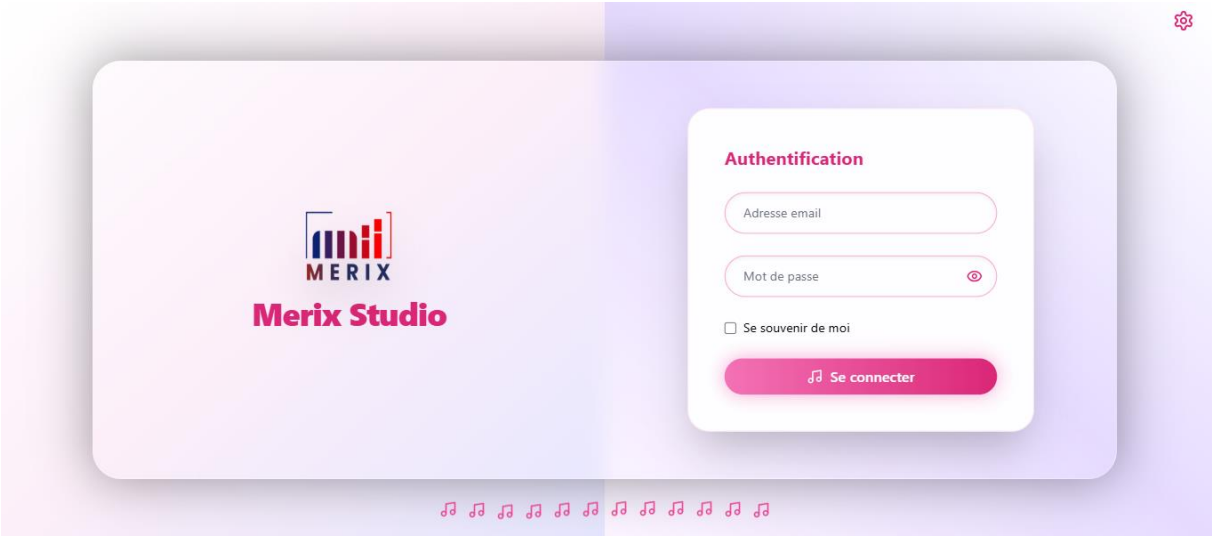

Figure 9.1 Interface authentification en mode clair

Cependant l'application a la fonctionnalité mode sombre qui s'applique à tous les pages, dont la figure suivante représente celle du page de connexion.

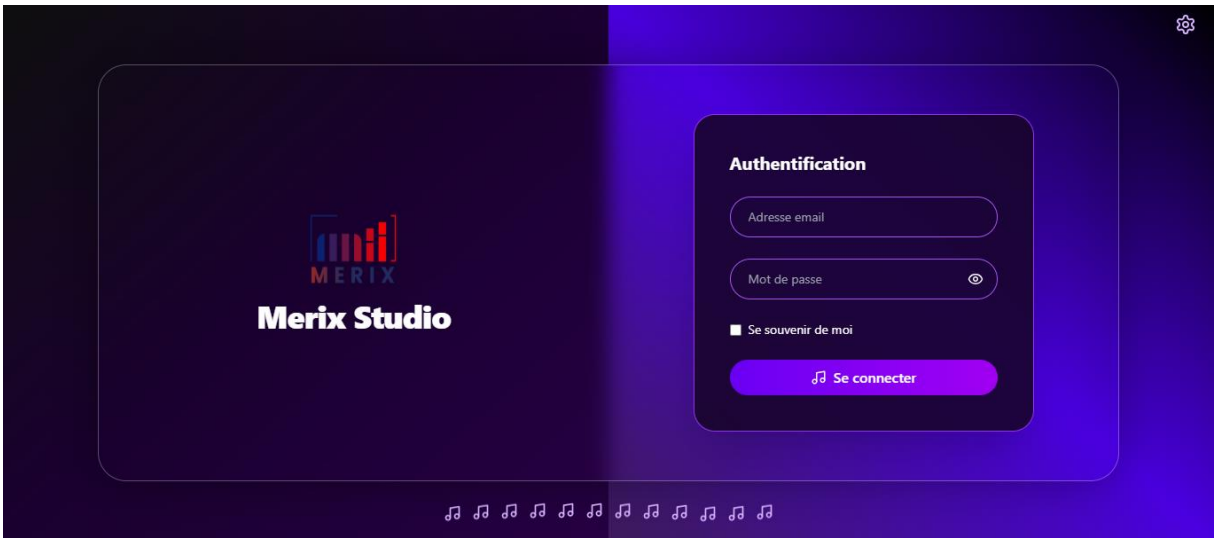

Figure 9.2 Interface de connexion en mode sombre

9.2 Présentation du page d'accueil

L'accueil héberge le recette annuelle et mensuel de l'application. Elle peut être filtré selon le souhait de l'utilisateur en mois ou en année ainsi que les deux en même temps.

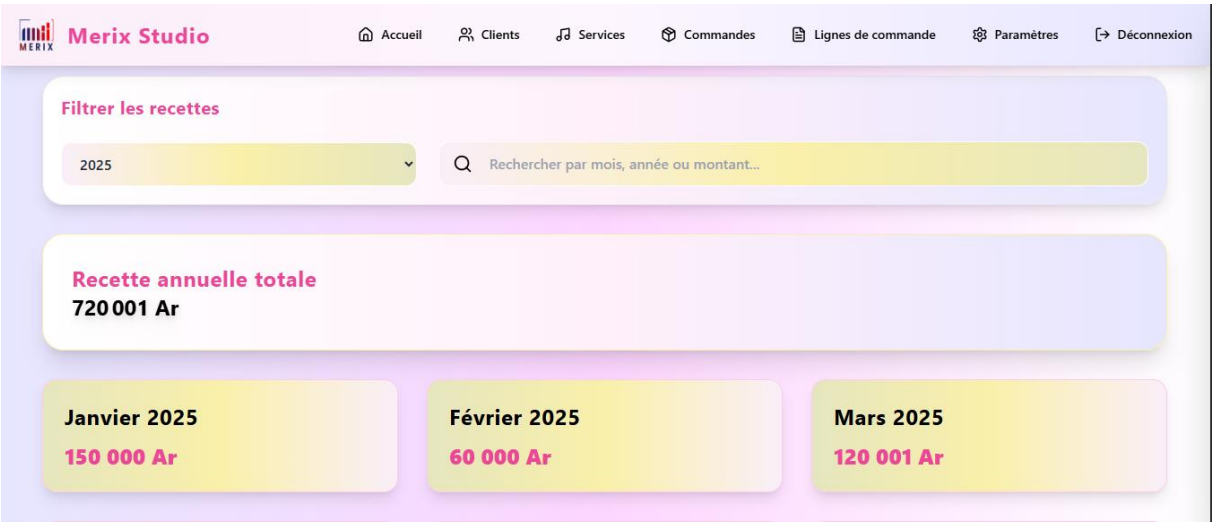

Figure 9.3 Recette annuelle et mensuelle

9.3 Présentation du CRUD

Parmi les fonctionnalités de l'application, le CRUD est l'un des fondamentaux. Il permet de faire la gestion des clients, services, les commandes ainsi que les lignes de commandes. L'interface ci-dessous représente le côté gestion des clients avec le filtrage dynamique tandis que les services, commandes et lignes de commandes ont le même.

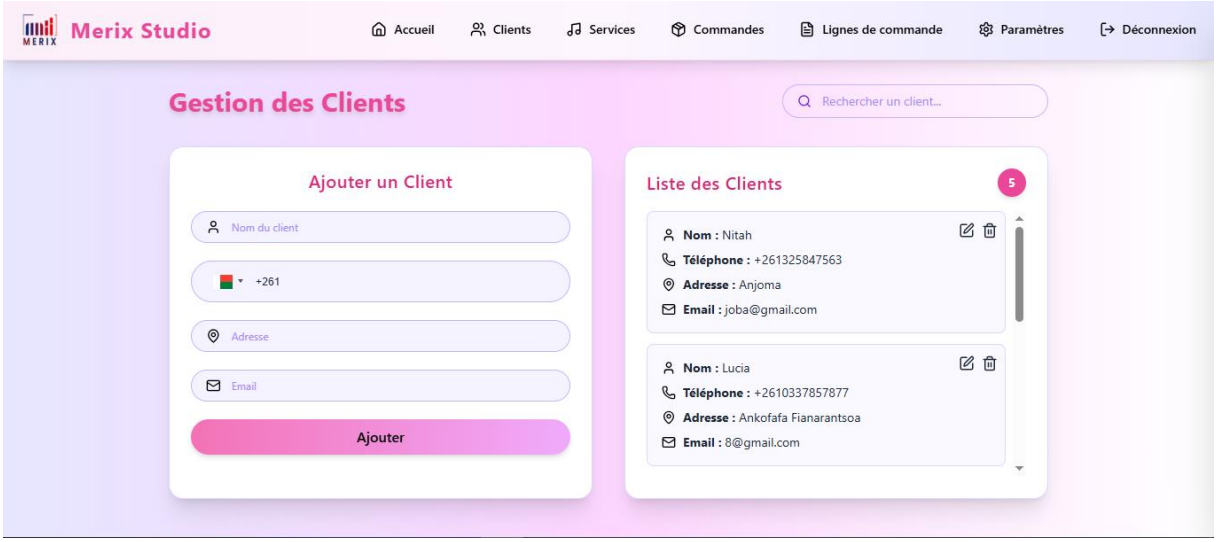

Figure 9.4 Interface représente le CRUD dans l'application

9.4 Présentation de facture

L'exportation PDF permet de télécharger en temps réel le facture de chaque client en format PDF. Pourtant, l'utilisateur peut gérer en même les déroulements des paiements pour données pour générer une facture en fonction de celui-ci.

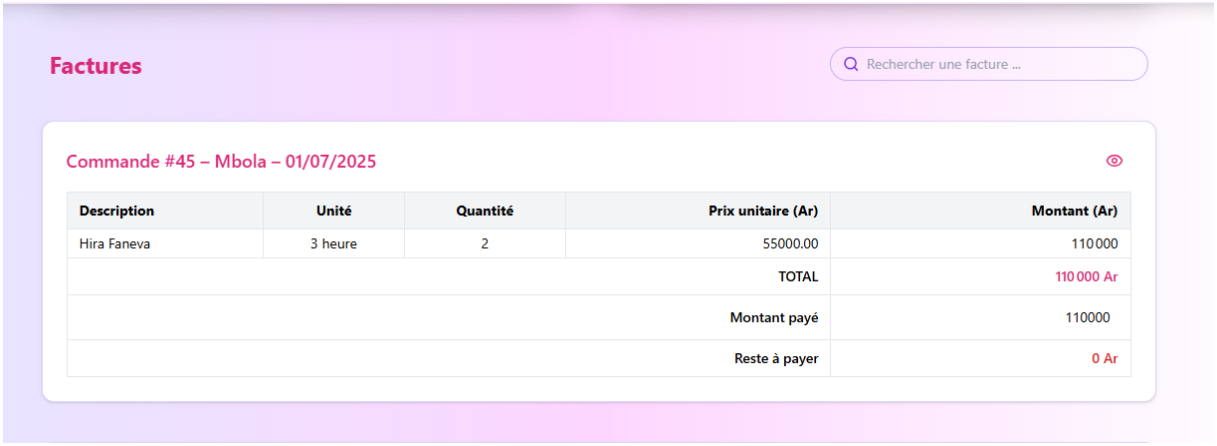

Figure 9.5 Extrait de la facture dans l'application

Comme toute application moderne et fonctionnelle, les informations de l'entreprise doit être requis dans chaque facture. Voici un extrait de code d'une facture télécharger en format PDF.

MERIX

Lot 2P 86 Tambohobe Nord
Fianarantsoa 301
Tél : 034 60 189 97
NIF: 5018431999
STAT: 46900232021000090

Facture n° : 00045
Date de facturation : 21/08/2025
Date de commande : 01/07/2025

Client : Mbola

Description	Unité	Quantité	Prix unitaire (Ar)	Montant (Ar)
Hira Faneva	3 heure	2	55000.00	110 000
			TOTAL	**110 000 Ariary**

Paiement complet

RAZAFISAMBATRA Tommy Gabrielo, Gérant
Signature

Figure 9.6 Extrait d'une facture en format PDF

9.5 Présentation des fonctionnalités supplémentaires

Le paramètre regroupe les fonctionnalités supplémentaires de l'application. Dont, il y a l'affichage des informations du studio qui se trouve dans chaque facture ainsi que l'affichage des listes de connexion sur l'application. Puis, la fonctionnalité qui permet de changer de mot de passe en plus de préférence qui permet de change de thème. Ensuite, la notification qui notifie les commandes enregistrer ou modifie du jours, et les à livrer ou à réaliser demain dont les marquées comme lue sont enregistrer temporairement pendant 15 jours. Enfin les historiques qui représentent les commandes livrées, réalisé par mois, ceux à réaliser et à livrer aussi suivi de la partie qui représente la commande du jour.

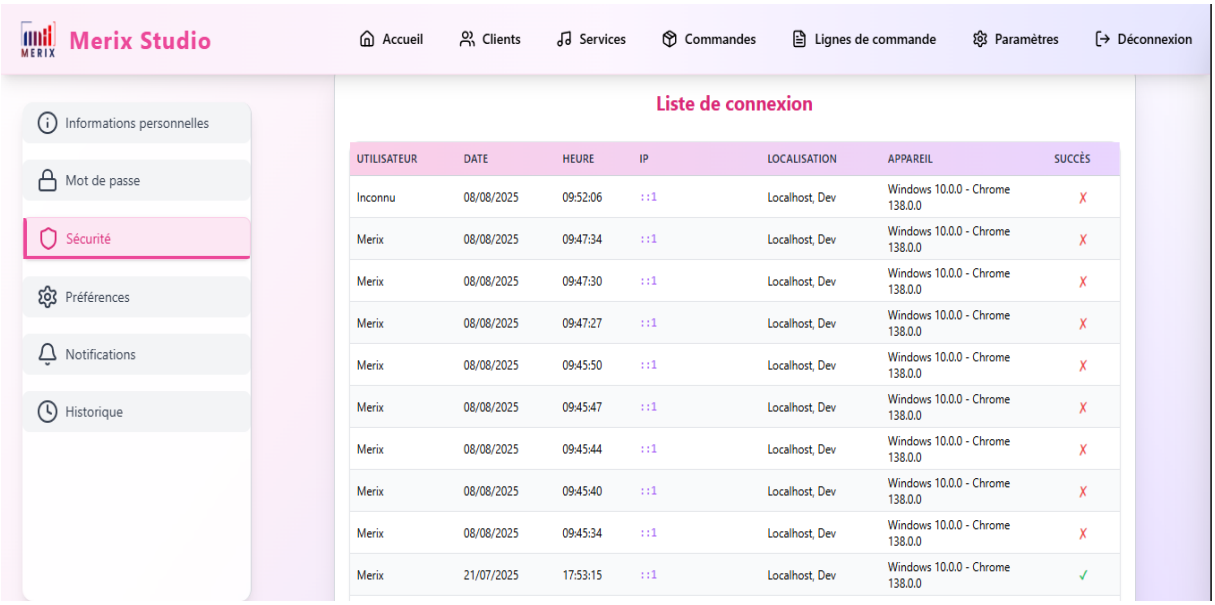

Figure 9.7 Interface du paramètre

CONCLUSION

Dans le cadre de ce projet, nous avons été confrontés à une problématique clairement identifiée : la nécessité de concevoir et de développer une application web moderne, performante et adaptée pour la gestion des services dans une maison de production audio. L'objectif principal était d'apporter une solution informatique efficace permettant de répondre de manière concrète à ces attentes, tout en respectant les contraintes techniques et organisationnelles du projet. La solution élaborée repose sur une architecture robuste, combinant des technologies récentes et largement adoptées dans le domaine du développement logiciel. L'application réalisée a su démontrer sa pertinence à travers ses fonctionnalités, sa facilité d'utilisation et sa capacité d'adaptation.

Parmi les points forts de l'application développée, il convient de citer sa convivialité, sa rapidité d'exécution, ainsi que la modularité de son code facilitant de futures évolutions. Le recours à des technologies modernes comme React.js, Node.js, Express, ainsi que l'intégration d'une base de données MariaDB via XAMPP, a permis d'assurer à la fois performance et maintenabilité. De plus, des optimisations notables ont été réalisées, notamment l'utilisation d'outils tels qu'Axios pour la gestion des requêtes HTTP et Concurrently pour une exécution simultanée fluide du frontend et du backend, améliorant ainsi l'efficacité du processus de développement.

Cependant, il est important de souligner certains points faibles. Quelques limites demeurent, telles que la dépendance aux environnements d'exécution choisis, ou encore l'absence de certaines fonctionnalités avancées qui pourraient être intégrées dans de futures évolutions. Tels que l'amélioration de la fonctionnalités notifications qui n'est pas vraiment au point ; l'ajout du cryptage des mots de passe côté backend en plus de renforcement de sécurité lors de l'utilisation de l'application. Par ailleurs, plusieurs difficultés ont été rencontrées au cours de la mise en œuvre : sur le plan logiciel, la gestion de la compatibilité entre les différentes bibliothèques et outils a nécessité des ajustements minutieux.

En définitive, ce projet constitue une expérience formatrice et enrichissante, tant sur le plan technique que sur le plan méthodologique. Il a permis de mettre en pratique les acquis théoriques, tout en développant un esprit d'analyse et de résolution de problèmes. Les choix technologiques effectués se sont révélés pertinents et garants de la qualité de la solution produite. Cette application peut servir de base solide pour des optimisations futures et l'ajout de fonctionnalités plus avancées, en vue d'une amélioration continue.

BIBLIOGRAPHIE ET WEBOGRAPHIE

Document en ligne (internet)

https://fr.react.dev/ : les documentations sur React.

https://youtu.be/a3nj66pzKJ0 : tutoriel sur l'utilisation de TS avec Node.js et Concurrently.

https://youtu.be/FDjJA4OJcto : tutoriel sur Git et GitHub.

https://youtu.be/-jnV9cNSCS8 : tutoriel sur les outils frontends.

ANNEXES

Annexe A : le lien du projet sur GitHub https://github.com/Lucia-RASOANIRINA/studio-gestion.git

Annexe B : l'image de Xampp

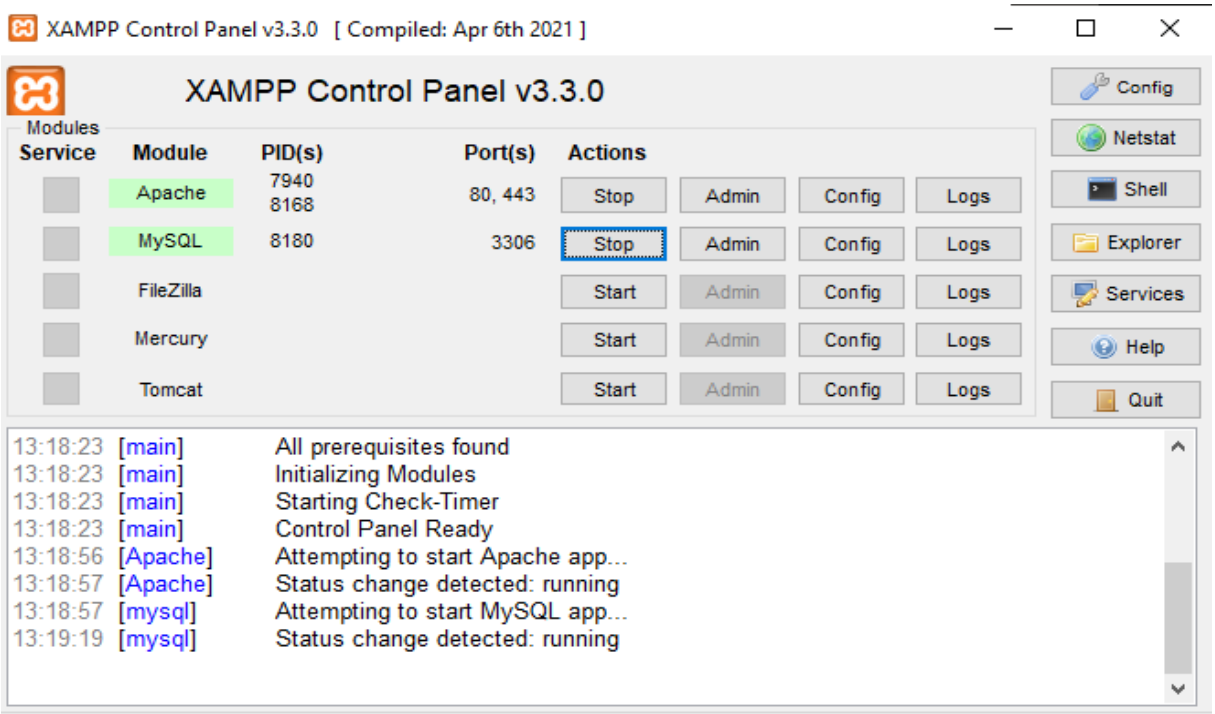

Figure : Image de Xampp